

ההבדל בין data mining ל text mining- ההבדל הוא סוג הנתונים. נתונים מובנים זה טבלה (Excel) עם עמודות ובעמודות יש נתונים מספריים קטגוריאליים. בצורה מובנה. כריית טקסט, מתעסקים בניתוח טקסטים, מה עושים ואיך מנתחים נתונים שאינם מובנים ומה התובנות שלהם.

דוגמא לתחומים:

- HR- ניתוח עובדים או מועמדים ומה הם חושבים על הארגון (לדוגמא glassdoor או לינקדין). התאמה בין קו"ח לבין תיאור משרה.
- שיווק- קמפיינים שיווקיים וקידום מוצרים מסוימים, איסוף טוויטים וניתוח שלהם. ניתן לבצע ניתוחים על הקמפיינים השיווקיים שלנו. ברגע שאפשר לנתח גם טקסטים יש לנו יתרון ולא צריכים לסמוך רק על נתונים מספריים. לדוגמא מס' הטוויטים בתחום מסוים. אלא להפיק תובנות גם רק מטקסט.
- בריאות- האם חולה חוזר לאשפוז כעבור 30 יום לדוגמא. האם החולה ימות במיון או תוך זמן מסוים.
- משפטים- שימוש בניתוח טקסט על תיקים מסוימים עם כמות גדולה של מסמכים.

NLP מול text mining- בכריית טקסט מתעסקים יותר בניתוח והוצאת תובנות מטקסט וב NLP מנסים לדמות התנהגות של בן אדם.

supervised vs unsupervised- ההבדלים ביניהם הוא שאלגוריתם supervised מציג בעיות שאנו יודעים את הפתרונות שלהם. לדוגמא כשיש סט נתונים עם תכונות של אובייקט מסוים ובסוף יש עמודה שיודעת לומר את הסוג של האובייקט. כלומר נדע מראש מה הסוג של האובייקט ויש לנו נתונים מתויגים. לתיג יש class, label, משתנה תלוי וכו'. ב 2 יש את זה בתוך עצי החלטות ורגרסיות. כלומר מאמנים את המכונה ללמוד ממה שקיים ודוגמאות קיימות ואחר כך נותנים לה נתונים חדשים שמקבלים תוצאות על סמך זה.

לעומת זאת unsupervised אין ממה ללמוד ואין מה לאמן. יש אוסף של נתונים ונרצה לחלק אותם לקבוצות (מסמכים דומים). לדוגמא אלגוריתם k-means.

שלב 1- הגדרת הבעיה והמטרות.

שלב 2- הגדרת הטקסט הרלוונטי לניתוח.

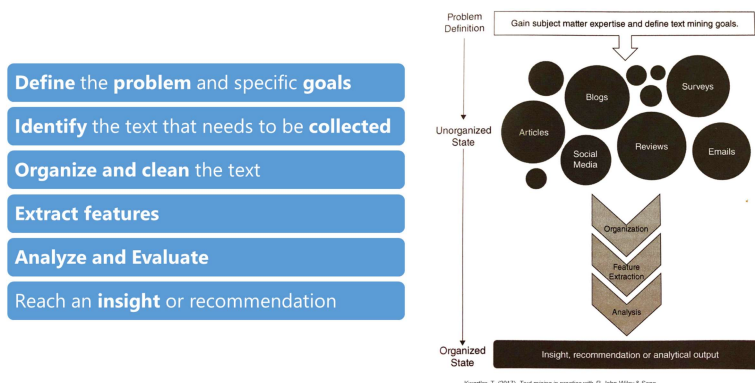
שלב 3- תמיד נתונים יגיעו "מלוכלכים" וצריך לדעת לנקות ולהכין את הטקסט.

שלב 4- מארגנים את הטקסט בצורה שנוכל לנתח אותו.

הפיצ'רים בשלב זה הם המשתנים הבלתי תלויים.

שלב 5- הערכה כמה הפיצ'רים טובים

שלב 6- מסקנות, תובנות, המלצות וכו'.



ידוגמא לניתוח מ-herb

בניתוח כמו פה, יש הרבה טקסט שהיה צריך לנקות, הרבה כפילויות של טקסט באותיות קטנות וגדולות, שם עצם ופועל שמדברים על אותו הדבר.

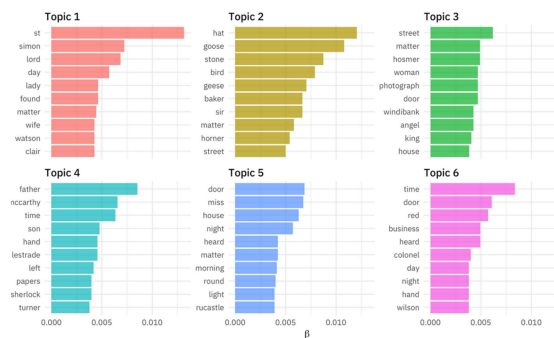
איך נדע אם לדוגמא לקצץ dry + drying לקטגוריה 1? תלוי בתוצאות, לכן יש חשיבות למובנות של האלגוריתם ומה מצופה ממנו.

Can I understand the main issues with this shampoo?

21 hair
4 product
4 scalp
4 made
4 dry
3 shampoo
3 tangled
3 frizzy
3 drying
3 super
3 fall
3 much
3 day



חלוקה ל Topics לפי unsupervised - חלוקה ללא הגדרה ברורה מראש.



Text Representation-

Bag of words - שיטה שלוקחת משפט מסוים ומתייחסת אליו כמו סל עם אוסף של מילים- אין משמעות לסדר בין המילים. לדוגמא יש 2 מסמכים הם המשפטים הבאים:

Document 1: John likes to watch movies. Mary likes movies too.

Document 2: John also likes to watch football games.

ייצוג מטריצוני על ידי וקטור:

Doc	John	likes	to	watch	movies	Mary	too	also	football	games
1	1	2	1	1	2	1	1	0	0	0
2	1	1	1	1	0	0	0	1	1	1

העמודות- הפיצ'רים שבונים. כל מילה מיוצגת על ידי עמודה. מספר עמודות יהיה המספר היוניקי שיש ב- 2 המשפטים. השורות- מספר המסמכים. שיטת ה term frequency - סופרים כמה פעמים כל מילה מופיעה.

Bag of N- grams - קיבוץ מילים בצורות שונות. כדי לשמור על הקונטקסט של המשפט נשתמש לדוגמא ב Big-grams.

Example -

Unigram - [let] [me] [explain] [with] [an] [example]

Bigram - [let me] [me explain] [explain with] [with an] [an example]

Trigram - [let me explain] [me explain with] [explain with an] [with an example]

לדוגמא: בדיקה בצמידים.

Doc	John likes	likes to	to watch	watch movies	movies Mary	Mary likes	likes movies	movies too	John also	also likes	watch football	football games
1	1	1	1	1	1	1	1	1	0	0	0	0
2	0	1	1	0	0	0	0	0	1	1	1	1

איך big grams שומר על הקונטקסט?

Document 1: **Lebron James** went to **high school**

Document 2: **James** Bay is very **high**

Unigram									
Doc	Lebron	James	went	to	high	School	Bay	is	very
1	1	1	1	1	1	1	0	0	0
2	0	1	0	0	1	0	1	1	1

Bigram									
Doc	Lebron James	James went	went to	to high	high school	James Bay	Bay is	is very	very high
1	1	1	1	1	1	0	0	0	0
2	0	0	0	0	0	1	1	1	1

בראשון יש דמיון ב-2 מילים ולכן מסיקים שיש דמיון בין המסמכים.

לעומת זאת ב big grams מסיקים שאין דמיון בין המסמכים בכלל.

האלגוריתם נותן שמירה על הקונטקסט.

הייצוג של הטבלה שהראינו עד כה - Document Term Metrix

ניתן להפוך את המטריצה לצרכים מסוימים.

TDM sparsity and Term sparsity

מטריצות שיש בהן הרבה אפסים. בגלל שיש מעט מילים במשפט. יש אלגוריתמי סיווג \ חיזוי שמותאמים יותר למטריצות מסוג זה. השאיפה לקבל מספר קטן כלומר יש פחות אפסים. פר מילה או פר מסמך - תלוי בנוסחה-

	1	2	3	4	5
R	0	0	1	1	0
Text	0	0	1	0	0
Stats	1	1	0	0	0
Mining	0	1	1	0	0
Book	0	0	0	0	1

$$\text{Sparsity}(TDM) = \frac{\text{Number of zeros}}{\text{Number of cells}}$$

$$\text{TDM sparsity} = 17/25 = 0.68$$

$$\text{The term "Mining" sparsity} = 3/5 = 0.6$$

$$\text{The term "Book" sparsity} = 4/5 = 0.8$$

$$\text{Sparsity}(Term) = \frac{\text{Number of zeros for the term}}{\text{Number of documents}}$$

Document- שורת טקסט שיש לחלק לחלקים קטנים יותר.

פונקציות בסיסיות:

```
setwd("E:/Text mining course/1")
```

#Set Options

Factor - הופך את הטקסט לפקטוריאלי. פקטור ב R זה משתנה קטגוריאלי (זכר- נקבה וכו').

במקרה זה ההגדרה היא false כלומר לא להפוך את הטקסט לקטגוריות.

```
options(stringsAsFactors=FALSE) #don't treat strings as factors (categories)
```

הגדרות מערכת שניתן להגדיר- בדוגמא זו- מוגדר כאן באנגלית של ארה"ב.

```
Sys.setlocale('LC_ALL','C')
```

טעינת ספריות שנקראות stringi + stringr- אם מקבלים שגיאה יש להתקין אותה לפני.

```
library(stringi) #load the library, if not exists please install
```

```
library(stringr) #load the library, if not exists please install
```

read csv

קריאת קובץ לתוך המערכת. שולחים את השם של הקובץ וניתן להוסיף פרמטרים נוספים.

```
text.df<-read.csv('oct_delta.csv') #read the csv file
```

```
head(text.df) #print 6 first rows
```

אם רוצים לגשת רק ל 4 העמודות הראשונות- לפני הפסיק מימד השורות ואחרי הפסיק מימד העמודות (נכתב כוקטור)

```
#head(text.df[,c(1:4)]) #first 4 columns
```

```
nrow(text.df) #number of rows
```

	weekday	month	date	year	text
1	Thu	Oct	1	2015	@ri...that can be frustrating..we hope to have yo...
2	Thu	Oct	1	2015	@rmarkerm Terribly sorry for the inconvenience. If we can b...
3	Thu	Oct	1	2015	@checho85 I can check, pls follow and DM your confirmati...
4	Thu	Oct	1	2015	@nealaa ...Alerts, pls check here: http://t.co/0jkZnT95Q *JH ...
5	Thu	Oct	1	2015	@nealaa ...advisory has only been issued for the Bahamas, b...
6	Thu	Oct	1	2015	@nealaa Hi. Our meteorologist team is aware of Hurricane J...

```
summary(text.df)
```

```
class(text.df$text)
```

```
head(text.df$text)
```

#basic text functions

מחרוזת- יצירת מחרוזות וספירת מספר התווים - התשובה תהיה 4.

nchar - number of characters

name <- "Roni"

nchar(name)

nchar(text.df\$text[1:10]) #number of characters in each text row

גישה למקום ספציפי בדטה- גישה לתא מסוים- שורה 4 עמודה 5.

text.df[4,5]

nchar(text.df[4,5]) #row 4 text length

אם נרצה להוסיף עוד עמודה- ניתן ליצור דרך dataframe- יש לשים בה את עמודת text ולספור אותה.

#data.frame(text = text.df\$text[1:10],nchar = nchar(text.df\$text[1:10]))

	text	nchar
1	@mjdout I know that can be frustrating.we hope to have yo...	119
2	@markerm Terribly sorry for the inconvenience. If we can b...	110
3	@checho85 I can check, pls follow and DM your confirmati...	78
4	@nealaa ...Alerts, pls check here: http://t.co/0jlcZnT95Q *JH ...	65
5	@nealaa ...advisory has only been issued for the Bahamas, b...	137
6	@nealaa Hi. Our meteorologist team is aware of Hurricane J...	142

mean(nchar(text.df\$text)) #mean of all text rows

remove blank documents

מחזיר אוסף מסוים של עמודות או שורות. תשאיר את מי שמספר התווים שלו גדול מ-0. הדאטה פרים החדש שלא כולל

תאים ריקים ייכנס לתוך משתנה שנקרא subset.doc

subset.doc <- subset(text.df, nchar(text.df\$text)>0)

וקטור עם סטרינגים מופרדים בפסיקים, מגדירים לו איך לשרשר. נשתמש לשרשור עמודות טקסט.

paste - concatenate strings

paste(c('a', 'b', 'cd'), collapse='|')

paste0('a', 'b', 'cd')

שליפה של חלק ממחרוזת.

substr - extract a sub-string in a character vector

str <- "abcdefg"

substr(str, 1, 4)

```
num <- 2
last <- substr(str, nchar(str)-num+1, nchar(str)) # two last characters
```

לוקחים את המחרוזת str ומהמקום ה-5 תעשה החלפה ותחליף את זה ב 3 נקודות. הוא מחליף רק עד מספר התווים שהיו קיימים (כלומר אם ממקום 5 יש 3 תווים או לא יוכל להכניס יותר מ 3 נקודות)

```
# substring - replace a sub-string in a character vector
```

```
substring(str, 5) <- c("...")
```

```
str
```

```
# grep - find
```

החלפת תו בתו אחר. רק כדי למצוא את המופע המיוחד שמחפשים. חפש את המילה England. החלק של

ignore.case = T מחזיר שקיים במשפט 1 ו-3. ignore.case מתעלם מאותיות קטנות\ גדולות.

```
sentences <- c("England is Beautiful", "just an example", "england left the EU")
```

```
grep("England", sentences, ignore.case = T)
```

מציאה והחלפה- למצוא מחרוזת ולהחליף במחרוזת אחרת.

```
# gsub - find and replace
```

תפעיל את הפונקציה- תחפש את England ותחליף ב UK.

```
sentence <- "England is Beautiful. england left the EU"
```

```
gsub("England", "UK", sentence)
```

הפיכת וקטור לרשימה.

```
# strsplit - split strings
```

```
x <- "get%better%every%day"
```

```
xs <- strsplit(x, split = "%")
```

הפיכת רשימה בחזרה לוקטור.

```
# turns to vector
```

```
unlist(xs)
```

יצירת פונקציה המחזירה את 2 התווים האחרונים (2 התווים של הנציג במקרה של הדוגמא בכיתה). הפונקציה מקבלת 2 ארגומנטים (text,num)- הטקסט וכמה נרצה לשלוף. הפונקציה מחשבת את ה last.

```
# agent workload
```

```
last.chars<-function(text,num){
```

```
  last<-substr(text, nchar(text)-num+1, nchar(text))
```

```
  return(last)
```

```
}
```

```
last.chars("R text mining is great",5)
```

```
last.chars(text.df$text[1:2],2)
```

```
table(as.factor(last.chars(text.df$text,2))) # /2 is a continuous message, ignore
```

Document Term Matrix

DTM

ייצוג של וקטור עם 2 איברים שנקרא טקסט.

לאחר שנטען את ספריית tm. יש פונקציות לעבודה עם טקסים. בספרייה זו יש לעבוד עם corpus (אוסף של מסמכים).

```
text <- c("This is a small example, the word small appears twice", "This is the second one")
```

```
# create corpus - vector source
```

VCorpus- תדע לקבל וקטור ודאטה פריים source.

```
corpus <- VCorpus(VectorSource(text))
```

```
# another option - create corpus from data frame source
```

```
# first column MUST be named doc_id
```

```
df <- data.frame(txt = text)
```

```
df <- data.frame(doc_id=seq(1:nrow(df)),text=df$txt)
```

ניתן ליצור גם corpus מתוך data frame.

```
corpus <- VCorpus(DataframeSource(df))
```

	doc_id	text
1	1	This is a small example, the word small appears twice
2	2	This is the second one

Document Term Matrix

DTM

DocumentTermMatrix- אובייקט מורכב שלא ניתן לראות במלואו. ברגע שנוצר הוא נותן כמה נתונים.

```
DTM <- DocumentTermMatrix(corpus)
```

```
as.matrix(DTM)
```

```
TDM <- TermDocumentMatrix(corpus)
```

```
as.matrix(TDM)
```

הערכים:

sparsity- 7/18. אחוז האפסים בנתונים.

```
<<DocumentTermMatrix (documents: 2, terms: 9)>>
Non-/sparse entries: 11/7
Sparsity           : 39%
Maximal term length: 8
weighting          : term frequency (tf)
> |
```

```
Docs appears example, one second small the this twice word
  1      1      1  0      0      2  1      1      1      1
  2      0      0  1      1      0  1      1      0      0
> as.matrix(DTM)
Terms
Docs appears example, one second small the this twice word
  1      1      1  0      0      2  1      1      1      1
  2      0      0  1      1      0  1      1      0      0
> view(df)
```

סוגי משקולות שונים - ניתן להגדיר את סוף המשקל גם כ default. ניתן לנרמל לפי frequency, binary

weightTf

```
DTM<-DocumentTermMatrix(corpus, control=list(weighting=weightTf))
as.matrix(DTM)
```

weightBin

```
DTM<-DocumentTermMatrix(corpus, control=list(weighting=weightBin))
as.matrix(DTM)
```

צמצום גודל המטריצה- מילים שהן לא חשובות. ישנם 4 משפטים שניצור ממנו corpus.

remove sparse terms

```
text <- c("This is a small example", "This is another example", "example appears many times", "small
example")
corpus <- VCorpus(VectorSource(text))
DTM <- DocumentTermMatrix(corpus)
inspect(DTM) # 1 -> 12, 0 -> 16. sparsity = 16/(12+16)
# Remove sparse terms from a document-term matrix.
smallDtm <- removeSparseTerms(DTM, .7) # 0.7 - the maximal allowed sparsity per term.
inspect(smallDtm)
```

מוריד את ה sparsity של כל מה שנמוך מ 0.7

```
> DTM <- DocumentTermMatrix(corpus)
> inspect(DTM)
<<DocumentTermMatrix (documents: 4, terms: 7)>>
Non-/sparse entries: 12/16
Sparsity           : 57%
Maximal term length: 7
Weighting          : term frequency (tf)
Sample            :
Terms
Docs another appears example many small this times
1      0      0      1      0      1      1      0
2      1      0      1      0      0      1      0
3      0      1      1      1      0      0      1
4      0      0      1      0      1      0      0
```

##Bi-grams (tm)

```
library(RWeka)
library(tm)
```

מעבירים מינימום ומקסימום של המילה- כלומר יהיו פה unigrams ו bigrams - הפונקציה תעשה טוקניזציה- תיצור עמודות ותפריד מסמכים לפי מה שיש ב BigramTokenizer.

```
BigramTokenizer <- function(x) NGramTokenizer(x, Weka_control(min = 1, max = 2))
DTM_BGram <- DocumentTermMatrix(corpus, control = list(tokenize = BigramTokenizer))
inspect(DTM_BGram)
```

Text Preprocessing-

היום נלמד כיצד מכינים טקסט לניתוח. המשימות הבסיסיות לכך: upper to lower, הורדת סימני פיסוק, הורדת מספרים, stemming (הפיכת המילה לצורת השורש שלה) ועוד..

TM Function	Description	Before	After
<u>tolower</u>	Makes all text lowercase	Starbuck's is from Seattle.	<u>starbuck's</u> is from <u>seattle</u> .
<u>removePunctuation</u>	Removes punctuation like periods and exclamation marks	Watch out! That coffee is going to spill!	Watch out That coffee is going to spill
<u>stripWhitespace</u>	Removes tabs, extra spaces	I like coffee.	I like coffee.
<u>removeNumbers</u>	Removes numerals	I drank 4 cups of coffee 2 days ago.	I drank cups of coffee days ago.
<u>removeWords</u>	Removes specific words (e.g. he, and, were & she) defined by the data scientists	The coffee house and barista he visited were nice, she said hello.	The coffee house barista visited nice, said hello.
<u>stemDocument</u>	Reduces prefixes and suffixes on words making term aggregation easier.	Transforming words to do text mining applications is often needed.	<u>Transform</u> word to do text <u>mine applic</u> is often needed.

RemoveWords - אם נעשה שליפה שכוללת את כל בתי המלונות בבוסטון- המילה בוסטון תופיע מלא פעמים. עם זאת היא מילה לא רלוונטית כי היא חלק מהחיפוש (אז זה ברור שהיא תופיע). לכן נכניס את המילה בוסטון מהרשימה שתתעלם ממנה.

StemDocument - הפעלת הפונקציה לדוגמא תשנה את transform > transforming. זה לא שורש אמיתי אלא שורש שמתכנתים כדי שתהיה אפשרות לחתוך את המילים.

יש רשימה של stopwords- באנגלית יש רשימות שאנשים קבעו כ stopwords- זו הרשימה הבסיסית-

```
> stopwords("english")
[1] "i" "me" "my" "myself" "we" "our" "ours" "ourselves"
[9] "you" "your" "yours" "yourself" "you" "him" "his" "his"
[17] "himself" "she" "her" "hers" "herself" "it" "its" "itself"
[25] "they" "them" "their" "theirs" "themselves" "what" "which" "who"
[33] "whom" "this" "that" "these" "those" "am" "is" "are"
[41] "was" "were" "be" "been" "being" "have" "has" "had"
[49] "having" "do" "does" "did" "doing" "would" "should" "could"
[57] "ought" "i'm" "you're" "he's" "she's" "it's" "we're" "they're"
[65] "i've" "you've" "we've" "they've" "i'd" "you'd" "he'd" "she'd"
[73] "we'd" "they'd" "i'll" "you'll" "he'll" "she'll" "we'll" "they'll"
[81] "isn't" "aren't" "wasn't" "weren't" "hasn't" "haven't" "hadn't" "doesn't"
[89] "don't" "didn't" "won't" "wouldn't" "shan't" "shouldn't" "can't" "cannot"
[97] "couldn't" "mustn't" "let's" "that's" "who's" "what's" "here's" "there's"
[105] "when's" "where's" "why's" "how's" "a" "an" "the" "and"
[113] "but" "if" "or" "because" "as" "until" "while" "of"
[121] "at" "by" "for" "with" "about" "against" "between" "into"
[129] "through" "during" "before" "after" "above" "below" "to" "from"
[137] "up" "down" "in" "out" "on" "off" "over" "under"
[145] "again" "further" "then" "once" "here" "there" "when" "where"
[153] "why" "how" "all" "any" "both" "each" "few" "more"
[161] "most" "other" "some" "such" "no" "nor" "not" "only"
[169] "own" "same" "so" "than" "too" "very"
```

כיצד נעשה זאת?



1. יש לומר מה ה source של input שלנו (וקטור של מסמכים או

dataframe) במקרה הזה יש לנו vector source ונקרא לו mysource

```
> # Make a vector source: my_source
```

```
> my_source <- VectorSource(my_tweets)
```

2. כך נוצר corpus ואפשר להפעיל עליו פונקציות.

```
> # Make a volatile corpus: my_corpus
```

```
> my_corpus <- VCorpus(my_source)
```

3. נגדיר איזה פונקציות נרצה שזה יפעיל-

```
> # Apply various preprocessing functions
```

- ניתן לה רשימת מילים שנרצה שירדו באובייקט שנקרא custom.stopwords.

```
> corpus <- tm_map(my_corpus, removeWords, custom.stopwords)
```

- נפעיל פונקציה שתסיר את אותן המילים.

```
> corpus <- tm_map(my_corpus, removePunctuation)
```

- נפעיל פונקציה שנקראת replace- היא לוקחת קיצורים של מילים ויודעת להחליף את זה למילה המקורית שלו בלי הקיצור.

```
> corpus <- tm_map(my_corpus, content_transformer(replace_abbreviation))
```

למה הסדר פה חשוב? למה לא נכתוב את זה בסדר שכתוב פה במצגת? כדי שלא יוסרו מילים שלא צריכות להיות מוסרות מכיוון שקוצרו לפני זה ואיבדו את משמעות המילה.

Stem vs. Root vs. Lemma

צורות להבאת הטקסט לצורתו הבסיסית.

Stem- קיצוץ התחלה או סוף של המילה (לפי החלטות של מתכנתים).

Root- שינוי של המילה (לפי הגדרות של בלשנות).

Lemma- מילה חוקית שחייבת להיות במילון.

Stem	Root	Lemma
The part of the word that is common to all its inflected variants	Primary lexical unit of a word, cannot be reduced into smaller constituents	The canonical form, dictionary form, or citation form of a set of words
stem + affixes = valid word	doesn't have to be a word	valid word
Example	Example	Example
Words: produce, production	Word: interrupt	Words: fly, flies, flew, flying
Stem = <u>produc</u>	Root = <u>rupt</u>	Lemma = fly

לכל אחת מהשיטות יש את המגבלות שלה.

> # Stem words

```
> stem_words<- stemDocument(c("complicatedly", "complicated", "complication"))
> stem_words
```

התוצאה: מציאת המכנה המשותף

```
[1] "complic" "complic" "complic"
```

Complete words - אחרי שמבצעים ניתוח ו stemming - קשה לקרוא את מה שיצא- לקחו פונקציה שמחזירה את ה stem למילה חוקית בשפה- מה שהיא עושה- היא לוקחת את המילה שעשינו לה stemming - נותנים לה מילון שהוא ה corpus בו השתמשנו ואנחנו מבצעים ממנו לבצע השלמה למילה חוקית בשפה.

נשתמש בזה אם נרצה להציג ב word cloud והמילים צריכות להיות חוקיות בשפה.

```
> # Complete words using single word dictionary
```

```
> stemCompletion(stem_words, c("complicate"))
      complic      complic      complic
"complicate" "complicate" "complicate"
```

בחלק הזה, נרצה להשתמש באוצר מילים של ה corpus- הוא יכול להתקל במילה מסוימת כמו למעלה עם 3 אופציות. הוא בודק מה המילה הכי נפוצה (לדוגמא complicated) ולפיה ישלים.

```
> # Complete words using entire corpus
```

```
> stemCompletion(stem_words, my_corpus)
```

פה רואים שזה לא מביא את המילים לצורתן המקורית.

> # Lemmatize words

```
> library(textstem)
```

```
> Vector <- c("complicatedly", "complicated", "complication")
```

```
> lemmatized_words <- lemmatize_words(vector)
```

מסתכל על הצורה המקורית ומתייחס אליה בצורה שונה (פועל, שם תואר וכו).

זה בעצם יתרון וחסרון כי בענן מילים כל 3 מילים יופיעו ויכול להיות שיש להם משמעות זהה.

אחד היתרונות הוא שאם יש מילה run ו ran (צורת עבר) שהם stem לא יזהה זהות דומות כמו פה.

```
> lemmatized_words
```

```
[1] "complicatedly" "complicate" "complication"
```

```
> # lemmatizing documents in a corpus
```

```
> corpus <- tm_map(corpus, lemmatize_strings)
```

בדקו כל מיני אלגוריתמים והתוצאות יצאו טובות יותר ב stemmer ולכן זו השיטה המקובלת.

Regular expressions [^ \ \s A-Z a-z \ ' \ \ -]

המטרה שלנו תהיה לכתוב כלל שיחליף רצף תווים ברצף אחר.

Regular Expression Syntax:

Syntax	Description
\d	Digit, 0,1,2 ... 9
\D	Not Digit
\s	Space
\S	Not Space
\w	Word
\W	Not Word
\t	Tab
\n	New line
^	Beginning of the string
\$	End of the string
\	Escape special characters, e.g. \ is "\", \+ is "+"
	Alternation match. e.g. /(e d)n/ matches "en" and "dn"
.	Any character, except \n or line terminator
[ab]	a or b
[^ab]	Any character except a and b
[0-9]	All Digit
[A-Z]	All uppercase A to Z letters
[a-z]	All lowercase a to z letters
[A-z]	All Uppercase and lowercase a to z letters
i+	i at least one time
i*	i zero or more times
i?	i zero or 1 time
i{n}	i occurs n times in sequence
i{n1,n2}	i occurs n1 - n2 times in sequence
i{n1,n2}?	non greedy match, see above example
i{n,}	i occurs >= n times
[:alnum:]	Alphanumeric characters: [:alpha:] and [:digit:]
[:alpha:]	Alphabetic characters: [:lower:] and [:upper:]
[:blank:]	Blank characters: e.g. space, tab
[:cntrl:]	Control characters
[:digit:]	Digits: 0 1 2 3 4 5 6 7 8 9
[:graph:]	Graphical characters: [:alnum:] and [:punct:]
[:lower:]	Lower-case letters in the current locale
[:print:]	Printable characters: [:alnum:], [:punct:] and space
[:punct:]	Punctuation character: ! " # \$ % & ' () * + , - . / : ; < = > ? @ [\] ^ _ { } ~
[:space:]	Space characters: tab, newline, vertical tab, form feed, carriage return, space
[:upper:]	Upper-case letters in the current locale
[:xdigit:]	Hexadecimal digits: 0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f

דוגמאות:

חיפוש בטקסט והחלפה במשהו אחר.

•gsub function - Replaces all matches of a string

•Example:

•gsub("https\\S*", "", tweets\$text)

•gsub("@\\S*", "", tweets\$text)

לפעמים פונקציות של ניקוי זה לא מספיק, נניח לא נרצה להוריד את כל סימני הפיסוק אלא להשאיר מקפים. גם לזה ניתן לכתוב פונקציות ניקוי.

[^ \ \s A-Z a-z \ ' \ \ -]

תמצא לי כל תו חוץ מהתווים הבאים: רווח, תווים בין A-Z ו a-z, גרש ומקף.

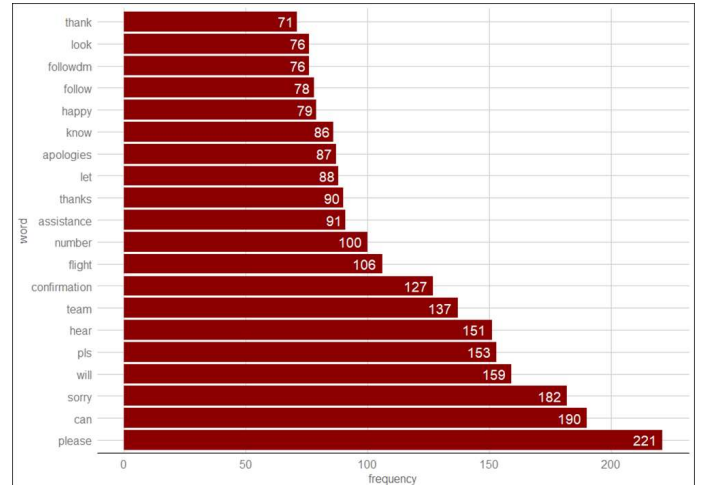
[^] – any character except
 \s – space
 A-Za-z – all the letters
 \' = '
 \\- = -

הצגת טקסט בויזואליזציות-

הבדל בין Comparison cloud ל Word cloud זה הצגה מול השוואה. Commonality cloud מציג דברים שומים בין מילים ו Polarized cloud מראה מרחק בין טקסטים שונים.

```
ggplot(freq.df[1:20,], aes(x=word,
y=frequency)) +
geom_bar(stat="identity", fill='darkred')
+ coord_flip()
```

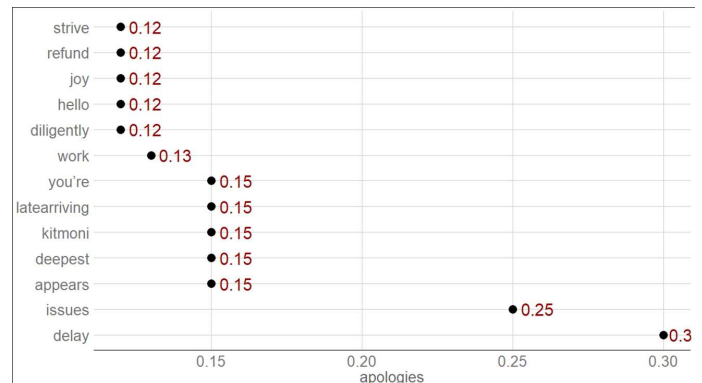
הפונקציה נקראת geombar שמציגים בה עד 20 מילים
(הצירים הפוכים אבל יש את הפונקציה coord_flip
שהופכת את התצוגה).



associations<-

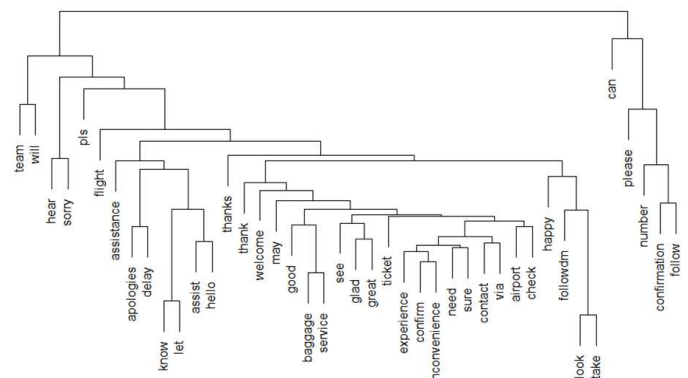
```
findAssocs(tdm, 'apologies', 0.11)
```

פונקציה שמוצאת אסוציאציה בין מילים. למשל רוצים לדעת על מה הנציגים של דלתא מתנצלים- נרצה למצוא איזה מילים נמצאות באסוציאציה גבוהה עם apologies. המספרים הם בין 0 ל-1 שמתארים קשר. ככל שהוא קרוב יותר ל-1 הקשר חזק יותר. ניתן לבחור ערך סף. לדוגמא פה זה מה שגדול יותר מ-0.11.



```
hc <- hclust(dist(tdm2, method = "euclidean"),
method = "complete")
plot(hc,yaxt='n',main = '@DeltaAssist Dendrogram')
```

צורה ייצוג שמראה שכל ששני איברים יותר קרובים אחד לשני זה אומר שהם מופיעים יותר ביחד. לדוגמא ניתן לסמן קו שיחלק את הטקסט שלנו לאשכולות. דוג: פונים לבוט שירות ולפי מספר פנייה הוא יודע לחלק את הניתוב של השיחה למקום הנכון (תסריט שיחה). השיטה תהיה complete- לא ניכנס לשוני בין השיטות



השוניות.

Commonality cloud

commonality.cloud(tdm.m, max.words = 200, random.order=FALSE, colors=pal)

מראה האם התצוגה תהיה רנדומלית, כמה מילים מופיעות וכו.

Comparison cloud

comparison.cloud(tdm.m, max.words = 200, random.order=FALSE, title.size = 1, colors=brewer.pal(ncol(tdm.m), "Dark2"))

#number of colors just like the number of different documents we compare

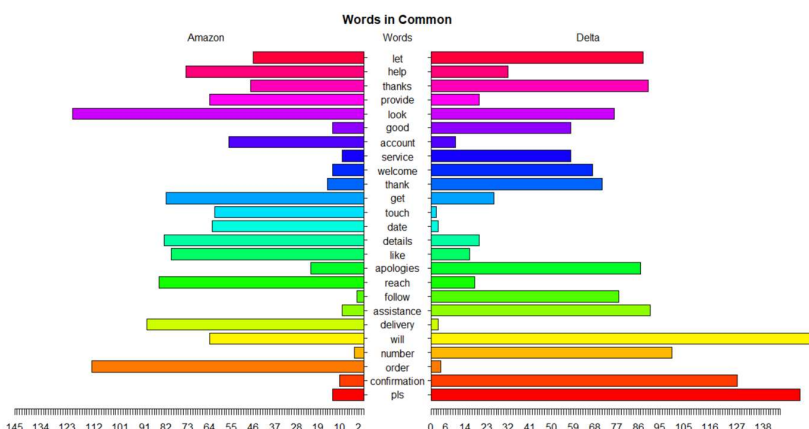
מבצעים השוואה בין שיחות של נציגי אמאזון ודלתא איירליינס וניתן לראות את ההבדלים במילים של כל אחד מהנציגות.

ענן המילים זה מקום טוב לראות אם הטקסט נוקה כמו שצריך. אם הטקסט לא נוקה טוב- נראה כל מיני סימני פיסוק, קישורים ודברים שיבלטו לעין.

האם מילה מסוימת יכולה להופיע גם וגם (בשני הצבעים)- היא תופיע רק בצד אחד (הפופולארי יותר)- במקרה הזה בצד של דלתא.

Polarized Tag Plot

נרצה לראות את ההבדלים בין התדירות של מילה מסוימת בין דלתא לאמאזון. למטה ההבדלים הגדולים ביותר ולאט לאט קטנים. ההבדל הכי גדול לדוגמא במילה pls (הכי למטה) נראה כי הנציגים של דלתא מרגישים יותר בנוח להשתמש בה מאשר אמאזון.



```
pyramid.plot(top25.df$x, top25.df$y, labels = top25.df$labels,
gap = 30, top.labels = c("Amazon", "Words", "Delta"),
main = "Words in Common")
```

Commonality Cloud - מאפשר לראות סטים של מילים ב- 2 קורפוסים. לדוג' נאום של ביבי מהפעם ה1 שנבחר לאחרונה. נוכל לייצר זאת ולראות את המילים שחזרו על עצמן.

Comparison cloud - לעומת זאת אם נרצה לראות את ההבדלים בין הנאומים נריץ את האלגוריתם ונראה את ההבדל.

ובדלתא. משתמשים בפונקציה ונותנים לה את ה dtm . גודל המילה מייצגת את כמות המילים שהיא מופיעה (לפי מינימום הפעמים באחד המסמכים- לדוגמא אם היא מופיע 100 פעמים בדלתא ו 99 פעמים באמאזון היא תופיע 99 פעמים), כלומר החיתוך בין שני המסמכים.

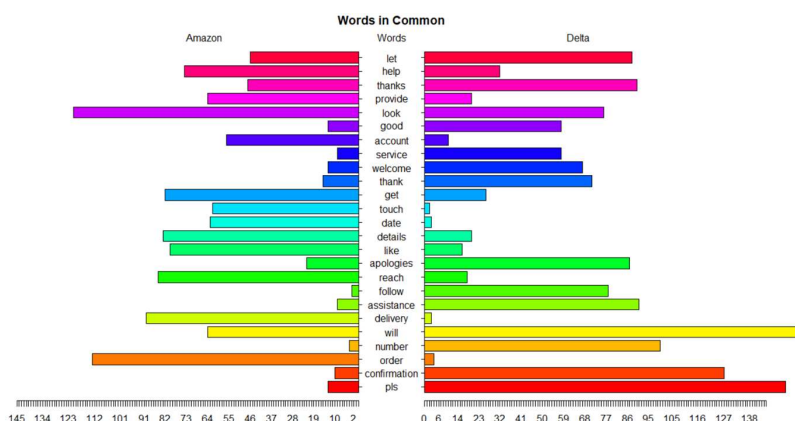
```
commonality.cloud(tdm.m, max.words = 200, random.order=FALSE, colors=pal)
```

הפונקציה מאפשרת דיפולט (min) אבל אם רוצים לקבל מקסימום או מספר שונה ניתן לשחק עם האלגוריתם.

החסרונות- אם ישנה מילה שמופיעה מספר פעמים מועט בדלתא והרבה באמאזון נקבל את המינימום.

Comparison cloud - אילו מילים מופיעות יותר בקורפוס אחד מבאחר.

```
comparison.cloud(tdm.m, max.words = 200, random.order=FALSE,  
title.size = 1, colors=brewer.pal(ncol(tdm.m), "Dark2"))
```



האם מילה יכולה להופיע ב 2 הקורפוסים?

כן, האלגוריתם נותן דגש האם מילה מסוימת מופיעה יותר בקורפוס אחד מאשר באחר.

Term Document Matrix (TDM)

Word	Doc1	Doc2
John	1	1
likes	2	1
to	1	1
watch	1	1
movies	2	0
Mary	1	0
too	1	0
also	0	1
football	0	1
games	0	1

כל מילה בכל אחד מהמסמכים מופיעה באחת השורות, עבור כל מסמך יש עמודה (יכול להיות גם tweet)

וסופרים ושמים במטריצה כמה פעמים כל מילה מופיעה.

שיוצרים TDM - יש פרמטר שנקרא waiting שמאופיין במשקלים - כאן $\text{weighting} = \text{weightTf}$

המשקלים הפופולריים:

Bag of words – Weight = Term frequency

first: This document is about a **cat**, the **cat** is the topic of this document

second: This document is about a **dog** the **dog** is the topic of this document

ניתן לראות שהמילה cat מופיעה פעמיים במסמך הראשון גם בטבלה. ולכן זה מראה על המשקלים של המילה.

לאחר מכן נבדוק על ידי סכימה מה חשיבות מילה מסוימת ב corpus.

במקרה הזה המילה document המופיעה בשני המסמכים - אם מילה מסוימת מופיעה בהרבה מאוד מסמכים כנראה שהיא פחות חשובה. יכול להיות בסיטואציה מסוימת למילה document תהיה בעלת חשיבות אם היא לא תופיע הרבה. לעומת זאת, המילה cat מופיעה רק במסמך 1 אבל מופיעה בו פעמיים. ולכן המילה cat נחשבת יחסית נדירה. למרות זאת בצורת הסכימה מקבלים שמילים כמו document או this יותר חשובות.

Weight = Term frequency

Terms	Docs
about	1 1
cat	2 0
document	2 2
dog	0 2
first	1 0
second	0 1
the	2 2
this	2 2
topic	1 1

Summarize rows

word	sum
about	2
cat	2
document	4
dog	2
first	1
second	1
the	4
this	4
topic	2

TF-IDF - motivation

This is a long document that contains many words, most of the words repeat in many documents but the word cat doesn't appear much. I saw a white cat today. The word cat appears 3 times in this document. What are the relevant words in this document? Can I determine it by word frequency?

The word 'word' appears many times in document 1 so we might conclude that document 1 is about words but actually the word 'word' appears in many documents so maybe it is not so relevant? Should we give it a high or a low weight?

המטרה היא להבליט (משקל גדול יותר) למילים שחוזרות על עצמן הרבה במסמך.

Document 1

Document 2

first: This document is about a **cat**, the **cat** is the topic of this document

second: This document is about a **dog** the **dog** is the topic of this document

Weight = TF-IDF

Terms	Docs		word	sum
	1	2		
about	0.00	0.00	about	0.00
cat	0.18	0.00	cat	0.18
document	0.00	0.00	document	0.00
dog	0.00	0.18	dog	0.18
first	0.09	0.00	first	0.09
second	0.00	0.09	second	0.09
the	0.00	0.00	the	0.00
this	0.00	0.00	this	0.00
topic	0.00	0.00	topic	0.00

פה רואים ש document מקבל 0 לעומת הדוגמא הקודמת cati מקבל משקל גבוה.

Bag of words – Weight = Binary (1 if appears, 0 otherwise)

לא אכפת כמה פעמים מילה מסוימת מופיעה, רק אם היא מופיעה או לא.

first: This **document** is about a **cat**, the **cat** is the topic of this **document**

second: This **document** is about a **dog** the **dog** is the topic of this **document, document, document, document, document, document**

Terms	Docs		word	sum
	1	2		
about	1	1	about	2
cat	1	0	cat	1
document	1	1	document	2
dog	0	1	dog	1
first	1	0	first	1
second	0	1	second	1
the	1	1	the	2
this	1	1	this	2
topic	1	1	topic	2

לפעמים יהיו בעיות , שרק בינארי יהיה מספיק (זה לא מראה על חשיבות). לדוגמא על דימיון בין מסמכים מסויימים. מה שחשוב לדעת זה אם מילה מופיעה או לא.

חישוב של TF- IDF

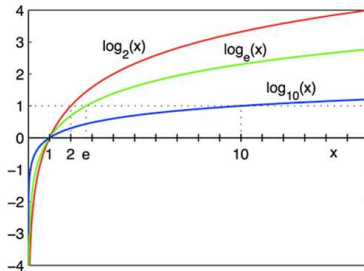
• חישוב- TF*IDF

• **TF = Term Frequency** (כמה פעמים היא מופיעה)

• **IDF = Inverse Document Frequency** - אם המילה נדירה זה מוסיף למשקל

המילה חלקי כלל המילים.

לוג סה"כ המסמכים בקורפוס חלקי כמה פעמים יש לי מסמכים שמכילים מילה מסוימת. זה פונקציה עולה. נקודת החיתוך היא ב $\log(1,0)$ של 1 זה 0. ככל שיש מספר יותר גדול, היא עולה ולאחר מכן מתייצבת.



בתוך הלוג יש סה"כ מסמכים (נגיד 100) חלקי לכמה פעמים מופיעה $(10) = 10$.
אם המילה מופיעה 20 פעמים $= 5$. כלומר מספר קטן יותר ולכן בהכפלה מכפילים
במספר יותר קטן = יותר נפוצה אבל פחות נדירה.

Number of terms in doc#1 = 200

$$\text{TF} = 5/200 = 0.025$$

Number of documents with the word 'kindle' = 10

$$\text{IDF} = \log_2(100/10) = 3.32$$

$$TF*|DF = 0.025*3.32 = 0.083$$

דגמל:

[illegible]

עבור המילה indle ב doc1 נרשום את התוצאה 0.083

שינוי מספר הפעמים שמילה מופיעה:

Number of documents with the word 'kindle' = ~~10~~ 20

$$\text{IDF} = \log(100/20) = 2.32$$

$$TF * IDF = 0.025 * 3.32 \quad 0.025 * 2.32 = 0.058$$

#Text mining in R

#create a function to clean the corpus

```
clean.corpus<-function(corpus){  
  
  corpus <- tm_map(corpus, content_transformer(tolower))  
  
  #corpus <- tm_map(corpus, removeWords, custom.stopwords)  
  
  corpus <- tm_map(corpus, removeNumbers)  
  
  corpus <- tm_map(corpus, removePunctuation)  
  
  corpus <- tm_map(corpus, stripWhitespace)  
  
  return(corpus)  
  
}
```

הפונקציה מחזיקה קורפוס נקי לתוך אובייקט קורפוס.

corpus	List of 2
\$ content:List of 2	
.. \$:List of 2	
.. .. \$ content: chr "first this document is about a cat the cat is the topic of this document"	
.. .. \$ meta :List of 7	
.. \$ author : chr(0)	
.. \$ datetimestamp: POSIXlt[1:1], format: "2022-04-29 07:48:26"	
.. \$ description : chr(0)	
.. \$ heading : chr(0)	
.. \$ id : chr "1"	
.. \$ language : chr "en"	
.. \$ origin : chr(0)	
.. attr(*, "class")= chr "TextDocumentMeta"	
.. .. attr(*, "class")= chr [1:2] "PlainTextDocument" "TextDocument"	
.. \$:List of 2	
.. .. \$ content: chr "second this document is about a dog the dog is the topic of this document"	
.. .. \$ meta :List of 7	
.. \$ author : chr(0)	
.. \$ datetimestamp: POSIXlt[1:1], format: "2022-04-29 07:48:26"	
.. \$ description : chr(0)	
.. \$ heading : chr(0)	
.. \$ id : chr "2"	
.. \$ language : chr "en"	
.. \$ origin : chr(0)	

weight by term frequency

tdm<-TermDocumentMatrix(corpus, control=list(weighting=weightTf)) #weightTf weights a
TDM by term frequency

```
inspect(tdm[1:9, 1:2]) #inspect the result
```

```
<<TermDocumentMatrix (terms: 9, documents: 2)>>
Non-/sparse entries: 14/4
Sparsity           : 22%
Maximal term length: 8
Weighting          : term frequency (tf)
Sample            :
Terms      Docs
  about    1 1
   cat     2 0
document   0 2
  dog      0 2
  first    1 0
  second   0 1
   the     2 2
  this     2 2
  topic    1 1
```

כמו שהרצנו במצגת בהתחלת השיעור.

```
tdm.m<-as.matrix(tdm) #convert to a matrix
```

לא רואים שינוי אבל הפיכת המטריצה מאפשרת פעולת sum.

```
term.freq<-rowSums(tdm.m) #summarize the rows (number of appearances of each term)
```

```
term.freq # term frequency
```

```
> term.freq
  about    cat document    dog    first    second    the    this    topic
    2      2      4      2      1      1      4      4      2
```

```
freq.df<-data.frame(word=names(term.freq),sum=term.freq) #create a dataframe with two
columns, word and sum
```

	word	sum
	about	2
	cat	2
	document	4
	dog	2
	first	1
	second	1
	the	4
	this	4
	topic	2

קוראים לעמודה הראשונה word ותיתן לה את השמות (הכותרות) והעמודה השניה היא sum. שם שמם את המספרים. מה שקורה זה שהוא יוצא df חדש.

```
wordcloud(freq.df$word,freq.df$sum, max.words = 10,min.freq=1, colors=c('black','darkred'))
```

מפעילה פונקציה- נותנים מילה ואת התדירות של המילה. מקס' 10 מילים ומינים מופיעות פעם 1



TF-IDF weight example

```
tdm.tfidf<-TermDocumentMatrix(corpus, control=list(weighting=weightTfIdf))
```

תואם למה שהופיע במצגת.

```
inspect(round(tdm.tfidf[1:9, 1:2],2)) #inspect the result
```

```
tdm.m<-as.matrix(tdm.tfidf)
```

Terms	Docs	
	1	2
about	0.00	0.00
cat	0.18	0.00
document	0.00	0.00
dog	0.00	0.18
first	0.09	0.00
second	0.00	0.09
the	0.00	0.00
this	0.00	0.00
topic	0.00	0.00

הופכים למטריצה על מנת לסכום

```
term.freq<-rowSums(tdm.m)term.freq[1:9]freq.df<-data.frame(word=names(term.freq),sum=round(term.freq,2))
```

```
#create a dataframe with two columns, word and sum
```

```
freq.df
```

```
wordcloud(freq.df$word,freq.df$sum, max.words = 10,min.freq=1, colors=c('black','darkred'))
```

dog
second
first
cat

binary weight example

	X...rid	text
1	1	first: This document is about a cat, the cat is the topic of this document
2	2	second: This document is about a dog the dog is the topic of this document, document, document, document, document, docum...

```
text.bin<-data.frame(doc_id=seq(1:nrow(text.bin.df)),text=text.bin.df$text)
```

לפני שיוצרים corpus מייצרים עמודת doc_id.

```
corpus.bin <- VCorpus(DataframeSource(text.bin))
```

מייצאים corpus ונותנים לו df.

```
corpus.bin<-clean.corpus(corpus.bin)
```

```
tdm.bin<-TermDocumentMatrix(corpus.bin, control=list(weighting=weightTf)) #weightTF - weight by term
```

```
frequency
```

נרצה לראות איך הענן מילים נראה עם משקל tf של ה corpus החדש.

```
<<TermDocumentMatrix (terms: 9, documents: 2)>>
Non-/sparse entries: 14/4
Sparsity           : 22%
Maximal term length: 8
weighting          : term frequency (tf)
```

```
inspect(tdm.bin[1:9, 1:2])
```

```
tdm.m<-as.matrix(tdm.bin) #convert to a matrix
```

```
term.freq<-rowSums(tdm.m) #summarize the rows
```

	Docs
Terms	1 2
about	1 1
cat	2 0
document	2 7
dog	0 2
first	1 0
second	0 1
the	2 2
this	2 2
topic	1 1

```
term.freq[1:9] # term weight
```

```
about      cat document      dog      first      second      the      this      topic
      2         2         9         2         1         1         4         4         2
```

```
freq.df<-data.frame(word=names(term.freq),sum=round(term.freq,2)) #create a dataframe with two columns,
word and sum
```

ה DF מכיל מילה ותדירות.

```
freq.df
```

```
wordcloud(freq.df$word,freq.df$sum, max.words = 10,min.freq=1,
colors=c('black','darkred'))
```



binary weight

כל השינויים שנעשה זה להחליף את פרמטר המשקל שמייצרים DocumentMatrix

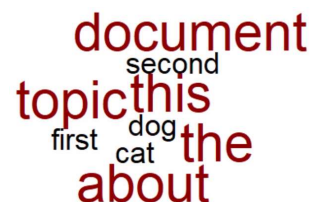
```
tdm.bin<-TermDocumentMatrix(corpus.bin, control=list(weighting=weightBin)) #weightBin - Binary weight
```

```
inspect(tdm.bin[1:9, 1:2]) #inspect the result
```

```
tdm.m<-as.matrix(tdm.bin) #convert to a matrix
```

```
term.freq<-rowSums(tdm.m) #summarize the rows
```

```
term.freq # term weight
```



```
freq.df<-data.frame(word=names(term.freq),sum=term.freq) #create a dataframe with two columns, word and
sum
```

```
freq.df
```

```
wordcloud(freq.df$word,freq.df$sum, max.words = 10,min.freq=1, colors=c('black','darkred'))
```

TF-IDF weight

```
tdm.bin<-TermDocumentMatrix(corpus.bin,
control=list(weighting=weightTfIdf)) #weight TF-IDF
```

```
inspect(round(tdm.bin[1:9, 1:2],4)) #inspect the result
```

```
tdm.m<-as.matrix(tdm.bin) #convert to a matrix
```

```
term.freq<-rowSums(tdm.m) #summarize the rows
```

```
term.freq # term weight
```

```
freq.df<-data.frame(word=names(term.freq),sum=round(term.freq,2)) #create a dataframe with two columns,
word and sum
```

```
freq.df
```

```
wordcloud(freq.df$word,freq.df$sum,max.words = 10,min.freq=1, colors=c('black','darkred'))
```

Terms	Docs	
	1	2
about	0.0000	0.0000
cat	0.1818	0.0000
document	0.0000	0.0000
dog	0.0000	0.1250
first	0.0909	0.0000
second	0.0000	0.0625
the	0.0000	0.0000
this	0.0000	0.0000
topic	0.0000	0.0000

מדוע יש משקל גדול יותר לכלבים ומחתולים למרות שמופיעים פעמיים במשפט?

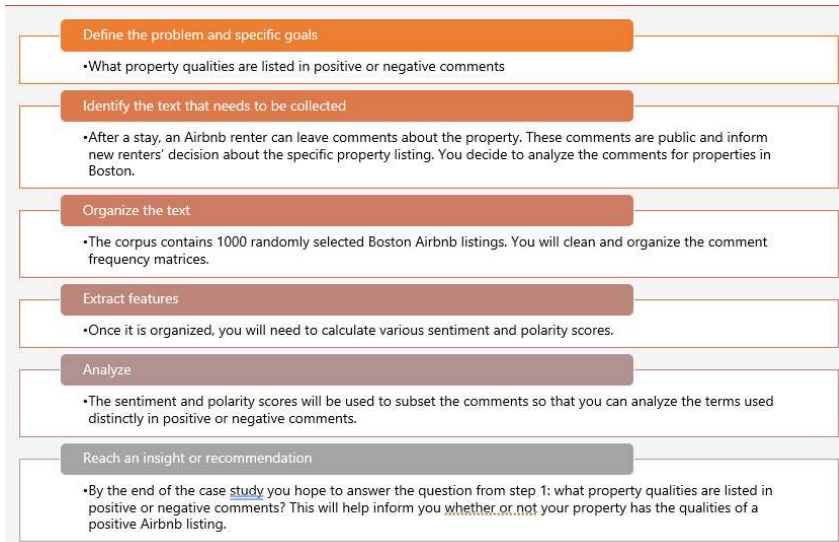
מכיוון שמשפט עם הכלב יש יותר מילים במשפט ולכן זה מקטין את המשקל שלו ביחס למשפט.

הסבר על הסיבה שסוכמים ב td-idf

אם יש לנו אוסף של כתבות- נרצה לקבץ ולמצוא מסמכים דומים אחד לשני. על מנת לבנות מערכת המלצה. נראה שלקוח קורא כתבה פוליטית. כל הכתבות יהיו בצורת וקטורים (מסמך). איך נשווה בין המסמכים? ונמדוד את המרחקים בניהם? נרצה למצוא מסמכים שמכילים מילים דומות. המילים בדומות יכולות להיות מילים נפוצות בשפה- לדוג' מילות קישור. כאן ניתן דגש בהשוואה בין 2 כתבות- זה לדוגמא "הוצאות בנט" ובמקום אחר יהיה "הוצאות" "בנט"- יש אפשרות להשוות בין 2 וקטורים במילים שבאמת חשובות בכתבה.

יש כמה סוגים של אלגוריתמים שמאפשרים למצוא רגש בטקסט, יש אלגוריתמים מבוססי מילון ומבוססי אלגוריתמי סיווג.

Use case - make sure my apartment has the qualities of a good rental-



1. מטרה- למצוא דירה טובה בבוסטון

2. איסוף מידע וביקורות

3. ניתוח ביקורות בין מה מאפיין ביקורות חיוביות למה מאפשר ביקורות שליליות.

למשל אפשר לעשות comparison cloud. ובסוף נגיע להמלצה או מסקנה

איך ניתן לחשב האם ביקורת מוסיימת היא חיובית או שלילית?

Subjectivity Lexicon

מילון דיפולטי שנותן ציון למילה- חיובי או שלילי.

Type	Length	Word	Part of Speech	Stemmed y/n	Polarity
Weak	1	Abundant	Adj	N	Positive
Weak	1	Abundance	Noun	N	Positive
Strong	1	Accede	Verb	Y	Positive
Weak	1	Accept	Verb	Y	Positive

**יש משמעות אם המילה מופיעה שפועל או שם עצם.

דוגמא לחישוב של polarity-

Word	Score
Good	+1
Challenging	-1

"Sentiment analysis in R is good yet challenging"

סה"כ הציון למשפט זה הוא 0 (מילה חיובית ומילה שלילית).

Valence shifters – "מחליפי תוקף"

I do like herring <-> I do not like herring $\neq$

1. **-negator** המילה not משנה את המשמעות. אם האלגוריתם נתקל ב not הוא יהפוך את הניקוד מ +1 ל -1.

I like mango <-> I really like mango $+$

2. **-amplifier** המילה really מחזקת את הלייק ובמקרה הזה נרצה לחזק את הציון ולתת במקום 1 לדוגמא 1.5.

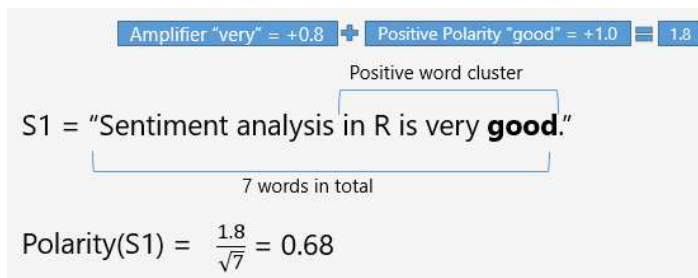
I like to drive <-> I hardly like to drive $-$

3. **-de- amplifier** המילה hardly - לא שלילית אבל עדיין מורידה ציון (מחליש את המילה).

(דוגמאות למילים בשקופית 12 במצגת).

חשוב לשים לה שלא מורידים את ה stop words שנותנים משמעות לרגש בניקיונות הטקסט.

Qdap's polarity function:



בספרייה זו, הוא עובר מילה מילה במשפט ומחפש מילים שנמצאות אצלו במילון.

במשפט המצורף הוא מוצא את המילה good. הוא מסתכל על 4 מילים לפני ואחרי המילה (במקרה הזה רק את 4 המילים לפני good) ונחפש במילים הקרובות האם יש מילים שמחזקות או מחליקות את good.

לדוגמא המילה very. תוסיף לציון עוד 0.8. וכעת הציון של המשפט הוא 1.8. לצורך החישוב ננרמל ונחלק ב שורש 7.

דוגמא נוספת:

```
updated_hash_sentiment <-
```

```
sentimentr::update_polarity_table(lexicon::hash_sentiment_jockers_rinker,
```

```
x = data.frame(words = c('be cleaner', 'repairs'), polarity = c(-1, -1), stringsAsFactors = FALSE)
```

```
)
```

```
> sentiment('This apartment can be cleaner')
```

element_id	sentence_id	word_count	sentiment
1:	1	1	5 0.2236068

```
> sentiment('This apartment can be cleaner',polarity_dt=updated_hash_sentiment)
```

element_id	sentence_id	word_count	sentiment
1:	1	1	5 -0.4472136

אחרי שהוספנו את המילון המעודכן הציון הופך לשלילי.

פונקציית sentiment תמיד מפרידה למשפטים ונותנת ציון לכל משפט בנפרד. אם משתמשים ב sentiment_by הציון יינתן לכל הקטע.

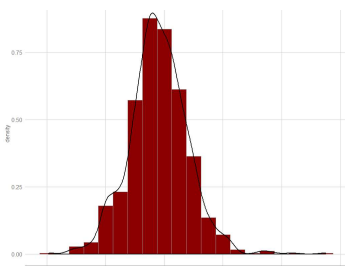
```
#calculate the polarity of each sentence
```

```
bos.pol<-sentiment(bos.airbnb$comments)
```

```
#calculate the polarity of each comment
```

```
bos.pol<-sentiment_by(bos.airbnb$comments)
```

Polarity distribution of Airbnb reviews



לקחנו אקסל גדול ועשינו ביקורות על דירות וחישבנו את הolarity. יצרנו על הניקוד היסטוגרמה. אנחנו רואים פה ממוצע וניתן לראות שמרבית הביקורות קיבלו ציון של מעל 1. כלומר מרבית האנשים אומרים גם משהו חיובי ולא רק משהו שלילי ולכן יש נטייה לכיוון של 1. מה שנראה שההיסטוגרמה נוצרה עם qdap. אם נרציץ עם sentimentR נראה ציון אחר יותר נמוך. אם נרצה למרכז את הביקורות נוכל לעשות זאת על ידי יצירת סטנדרטיזציה עם ממוצע 0. ונראה שכל מיני ביקורות שקיבלו 0.1-0.2 יומרו להיות ביקורות שליליות. כך נוכל לראות ביקורות חיובית כ- כל מה שגדול מ 0. אם לא נרצה לנרמל ולהשאיר את הדברים כמו שהם, נהיה צריכים להגדיר ערך סף, לפי הנתונים עצמם.

part 3.2 code Sentiment Scoring

```
setwd("E:/Text mining course/3")
options(stringsAsFactors=FALSE)
Sys.setlocale('LC_ALL','C')
polarity calculation
```

התקנת הספרייה

```
library(sentimentr)
```

example

1. המילון לא מכיר את ROFL כי זה סלנג. אז הציון יהיה 0 כי הוא לא מכיר את זה.

מה ניתן לעשות? אפשר להוסיף את המילה למילון שלנו.

```
sentiment('ROFL, look at that!') # ROFL =
Rolling On the Floor, Laughing
```

```
> sentiment('ROFL, look at that!')
  element_id sentence_id word_count sentiment
1:          1           1           4         0
```

2. מילים כמו wow מעלים את הציון של המשפט.

```
sentiment('wow, this is an excellent
restaurant')
```

```
> sentiment('wow, this is an excellent restaurant')
  element_id sentence_id word_count sentiment
1:          1           1           6 0.6123724
```

3. ברגע שמורידים את ה wow יתן לראות שגם הציון של המשפט יורד.

```
sentiment('this is an excellent restaurant')
```

```
sentiment('this is an excellent restaurant')
  element_id sentence_id word_count sentiment
1:          1           1           5 0.4472136
```

4. המילה bad מביאה אותנו לציון שלילי.

```
sentiment('this is a bad restaurant')
```

```
sentiment('this is a bad restaurant')
  element_id sentence_id word_count sentiment
1:          1           1           5 -0.3354102
```

comma

פסיקים, יש לעשות את הניקיונות בצורה שונה ממה שעשינו עד כה.

1. במשפט הראשון יש משמעות לפסיקים.

```
sentiment('this is a very good restaurant, I like it')
```

```
sentiment('this is a very good restaurant, I like it')
  element_id sentence_id word_count sentiment
1:          1           1           9 0.6166667
```

2. הדוגמא השניה מציגה משפט זהה אבל ללא פסיקים very מפיע על like. ברגע ששמים פסיק זה משפיע על

המשפט ומקטין את האשכול.

```
sentiment('this is a very good restaurant I like it')
```

```
sentiment('this is a very good restaurant I like it')
  element_id sentence_id word_count sentiment
1:          1           1           9 0.75
```

n.before

1. כמה מילים לוקחים לפני - מה שקורה זה שהציון של הסנטימנט יורד. המילה like והסנטימנט לא יושפע מ very.

```
sentiment('this is a very good restaurant and I like it', n.before = 2)
```

element_id	sentence_id	word_count	sentiment
1	1	10	0.7115125

```
sentiment('this is a very good restaurant and I like it') # 'very' affects 'like' (n.before = 5)
```

question

1. משפטים שמסתיימים בסימן שאלה לרוב זה לא משהו שמכיל בתוכו סנטימנט. למשל במשפט שמופיע. הוא

בודק אם יש לייק? ונותן סנטימנט חיובי, אבל אם מוסיפים את הפרמטר `question.weight = 0` הוא נותן סנטימנט 0.

```
sentiment('do you like it?') # default - question.weight = 1
```

```
sentiment('do you like it?', question.weight = 0) # assumption - pure questions are not polarized
```

element_id	sentence_id	word_count	sentiment
1	1	4	0.25

element_id	sentence_id	word_count	sentiment
1	1	4	0

like - non-verb form

ל like יכולים להיות תפקידים שונים במשפט. אם הlike הוא לא verb (אני אוהבת את המסעדה הזאת..) אלא משפטים כמו you are like me. החוקים אומרים שאם הlike מגיע לפני מילים מסוימות הוא לא נחשב פועל ואם נריך את הסנטימנט על משפט כזה נקבל 0.

```
sentiment('you are like me', neutral.nonverb.like = TRUE)
```

sentiment vs. sentiment_by

עד עכשיו השתמשנו ב sentiment - ישנם משפטים שהם text כמו המשפט פה: יש באמצע נקודה- מצד אחד הדירה היתה מלוכלכת אבל אהבתי מאוד את המטבח. כלומר יש פה משפט משולב. אם נריך את הפונקציה היא תפרק את המשפטים והיא תיתן פלט של אלמנט אחד ובתוכו יש 2 משפטים- אחד חיובי והשני שלילי.

```
sentiment('the apartment was dirty, I hardly could sleep at night because of the noise. Although I really liked the kitchen.')
```

element_id	sentence_id	word_count	sentiment
1:	1	14	-0.2538982
2:	1	2	0.6225786

נרצה לחשב ציון 1- שכולל את כל הקטע וברגע שנפעיל את sentiment_by היא תחשב את הממוצע

```
sentiment_by('the apartment was dirty, I hardly could sleep at night because of the noise. Although I really liked the kitchen.')
```

element_id	word_count	sd	ave_sentiment
1	20	0.6197627	0.1843402

average_weighted_mixed_sentiment - שכמחשבים ממוצע, תיתן משקל יותר גדול למשפטים שליליים. מכיוון שהנטייה של אנשים שהם כותבים ביקורת זה תמיד להכניס גם משהו חיובי. ולכן הציון הוא שלילי.

```
sentiment_by('the apartment was dirty, I hardly could
sleep at night because of the noise. Although I really
liked the kitchen.', averaging.function =
average_weighted_mixed_sentiment)
```

element_id	word_count	sd	ave_sentiment
1	20	0.6197627	-0.196507

פונקציה נוספת נקראת `extract_sentiment_terms`, מריצים את הפונקציה על המשפט והוא בעצם ייתן לי איזה מילים הוא זיהה במילון.

```
extract_sentiment_terms('the apartment was dirty, I
hardly could sleep at night because of the noise.
Although I really liked the kitchen.')
```

element_id	sentence_id	negative	positive
1:	1	1	dirty, noise
2:	1	2	liked

יש דרך אם נרצה לזהות משפטים חיוביים ומשפטים שליליים ולא מפריע לי שזה באותה ביקורת. מה שחשוב י זה לשהות על מה אנשים מתלוננים ונרצה לרכז ביחד את כל המשפטים השליליים והחיוביים. מה שנוכל לעשות זה לייצר וקטור שמכיל אוסף של משפטים. הם מופרדים בפסיק. נפעיל את הפונקציה `get_sentences` שתפריד את האלמנטים ותדע לפרק את המשפטים.

```
text <- c('the apartment was dirty, I hardly could sleep at night because of the noise. Although I really
liked the kitchen.', 'another text')
```

```
textSentences <- get_sentences(text)
sent <- sentiment(textSentences)
```

```
[[1]]
[1] "the apartment was dirty, I hardly could sleep at night because of the noise."
[2] "Although I really liked the kitchen."

[[2]]
[1] "another text"

attr(,"class")
[1] "get_sentences" "get_sentences_character" "list"
```

```
sent.df <- data.frame(text =
unlist(textSentences), sent)
```

	element_id	sentence_id	word_count	sentiment
1:	1	1	14	-0.2538982
2:	1	2	6	0.6225786
3:	2	1	2	0.0000000

ה DF, ישים את הטקסט של `textSentences` ויחבר את האובייקט `sent`

	text	element_id	sentence_id	word_count	sentiment
1	the apartment was dirty, I hardly could sleep at night because of the noise.	1	1	14	-0.2538982
2	Although I really liked the kitchen.	1	2	6	0.6225786
3	another text	2	1	2	0.0000000

כעת, נראה איך ניתן להוסיף מילים, כשנסתכל על תוצאות נראה שיש ביקורות שליליות שהאלגוריתם לא תפס. נרצה להוסיף מילים במקרים כאלה. דבר ראשון, נראה איך ניגשים למילון. המילון הדיפולטי נקרא `hash_sentiment_jockers_rinker` (כל מילה והניקוד שלה)

```
lexicon::hash_sentiment_jockers_rinker[1:30] #different weights
```

```
lexicon::hash_sentiment_jockers_rinker["hairy"]
```

```
lexicon::hash_sentiment_huliu["hairy"]
```

```
> lexicon::hash_sentiment_jockers_rinker["hairy"]
      x      y
1: hairy -0.1
```

דוגמאות נוספות- שלא קיים להן ציון:

```
lexicon::hash_sentiment_jockers_rinker["be cleaner"]  
lexicon::hash_sentiment_jockers_rinker["repair"]
```

```
> lexicon::hash_sentiment_jockers_rinker["repair"]  
      x y  
1: repair NA
```

כיצד נוסיף מילים למילון: updated_hash_sentimen

```
library(lexicon)
```

```
updated_hash_sentiment <-
```

```
sentimentr::update_polarity_table(lexicon::hash_sentiment_jockers_rinker,  
  x = data.frame( words = c('be cleaner', 'repairs'), polarity = c(-1, -1),  
    stringsAsFactors = FALSE))
```

שמרצים את הסנטימנט יוצא ציון חיובי, זה קורה כי לא אמרנו לו להשתמש בלקסיקון החדש שיצרנו (בדיפולטי המילה 'cleaner' זה חיובי)

```
sentiment('This apartment can be cleaner')
```

```
> sentiment('This apartment can be cleaner')  
  element_id sentence_id word_count sentiment  
1:          1           1          5 0.2236068
```

כאן אנו כבר אומרים לו להשתמש במילון החדש. והציון הופך שלילי.

```
sentiment('This apartment can be cleaner', polarity_dt=updated_hash_sentiment)
```

```
      sentiment('This apartment can be cleaner', polarity_dt=updated_hash_sentiment)  
  element_id sentence_id word_count sentiment  
1:          1           1          5 -0.4472136
```

13: didnt 1
14: doesn't 1
15: doesnt 1
16: don't 1
17: dont 1
18: hadn't 1
19: hadnt 1
20: hasn't 1
21: hasnt 1
22: haven't 1
23: havent 1
24: isn't 1
25: isnt 1
26: mightn't 1
27: mightnt 1
28: mustn't 1

#valence_shifters

מחלפי תוקף- איך נדע אילו מילים קיימים שמחליפות תוקף? קיים מילון שיש לו מילה ועמודת קוד מספרי
negator/ ampliphier האם זה

```
lexicon::hash_valence_shifters
```

#negators

```
lexicon::hash_valence_shifters[y == 1]
```

#amplifiers [intensifiers]

```
lexicon::hash_valence_shifters[y == 2]
```

#de-amplifiers [downtoners]

```
lexicon::hash_valence_shifters[y == 3]
```

emotion terms

נרצה למצוא את הרגש עצמו, האם הוא מכיל עצב, שמחה, הפתעה.

נייצר וקטור שנקרא טקסט ומכיל משפטים. ככל שמתקדמים המשפטים הופכים להיות יותר שליליים.

הבעיה באלגוריתם שהוא לא מבין את ה negators, לדוגמא ש not happy זה שלילי.

```
text <- c(
  "I love it",
  "I'm very happy right now",
  "I am not afraid of you",
  "I'm not angry with you",
  "I went to the terrible store",
  "I hate it when you lie to me, it's so humiliating",
  "I'm not happy anymore, it's time to end it",
  "I'm unhappy, want to die",
  "He is killing me")
```

לאחר שיוצרים את הטקסט נריץ פונקציה שנקראת `extract_emotion_terms`. התייחסות לקידומת חט כמשהו שלילי.
`extract_emotion_terms(text, emotion_dt = lexicon::hash_nrc_emotions, un.as.negation = TRUE)`

```
plot(emotion(text))
```

	element_id	sentence_id	anger	anticipation	disgust	fear	joy	sadness	trust	surprise
1:	1	1	I				love			<NA>
2:	2	1		happy			happy		happy	<NA>
3:	3	1				afraid				<NA>
4:	4	1	angry		angry					<NA>
5:	5	1	terrible	store	terrible	terrible		terrible		<NA>
6:	6	1	hate, lie		hate, humiliating, lie	hate		hate, lie		<NA>
7:	7	1		happy, time			happy		happy	<NA>
8:	8	1	unhappy		unhappy	die		die, unhappy		<NA>
9:	9	1	killing			killing		killing		<NA>

מתוך case study של Airbnb Boston

טעינת הרשומות לתוך R:

```
library(ggplot2)
library(ggthemes) #has predefined themes and color palettes for ggplot2 visualizations
library(sentimentr)
```

נוכל לבצע ניקיונות לפני שמבצעים סנטימנט אבל צריך להיזהר איתם. הניקיונות שכן נוכל לעשות הם:
 links and http, upper to lower cases, numbers
חשוב - לא לנקות סימני פיסוק, מילות עצירה.

```
bos.airbnb<-read.csv('bos_airbnb_1k.csv')
```

על מנת לנפעיל את ה `sentiment_by` על סט נתונים שלם.

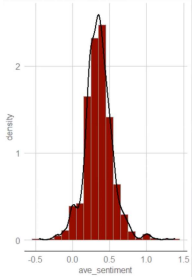
```
bos.pol.sentimentr<-sentiment_by(bos.airbnb$comments)
```

האובייקט החדש:

```
head(bos.pol.sentimentr) #save_sentiment is
the average polarity score
```

	element_id	word_count	sd	ave_sentiment
1:	1	77	0.2158782	0.3811683
2:	2	78	0.3690443	0.4250953
3:	3	39	0.1529963	0.2418153
4:	4	101	0.2698857	0.2214252
5:	5	16	0.3590662	0.2770975
6:	6	79	0.1050783	0.1464980

ניצור היסטוגרמה: התפלגות הסנטימנט הממוצע



```
ggplot(bos.pol.sentimentr, aes(x=ave_sentiment, y=..density..))+ theme_gdocs() +
#..density..is a computed variable, used to calculate the density of x, part of ggplot2.
  geom_histogram(binwidth=.1,fill="darkred",colour="grey60", size=.2) +
  geom_density(size=.75)
```

ניתן לעשות scaling- כלומר לעשות סטנדרטיזציה שממקצת את הכל לממוצע 0. ניצור עמודה חדשה ב

DF שנקראת sentiment_scaled. מצד שני נוסף עוד עמודה שנקראת sentiment שמכילה את ה score המקורי.

```
bos.airbnb$sentiment_scaled<-scale(bos.pol.sentimentr$ave_sentiment)
```

```
bos.airbnb$sentiment<-bos.pol.sentimentr$ave_sentiment #not scaled
```

```
table(sign(bos.airbnb$sentiment_scaled))
```

נבדוק כמה ביקורות חיוביות ושליליות יש. פונקצית sign תחזיר 1,0,-1 לפי סוגי הביקורות.

```
table(sign(bos.airbnb$sentiment)) #not scaled
```

```
> table(sign(bos.airbnb$sentiment_scaled))
-1    1
509 491
> table(sign(bos.airbnb$sentiment))
-1    0    1
28   17  955
```

מה שזה אומר שכנראה בתוך ה 1 מתחבאות גם ביקורות שליליות בגלל שלאנשים יש נטייה להוסיף משהו טוב בביקורות שליליות. וגם כי לא הוספנו את הפרמטר שמוסיף משקל גבוה יותר לשלילי. ניקח רק את מי שהסנטימנט שלו שלילי.

```
bos.airbnb[bos.airbnb$sentiment < 0, "comments"]
```

נשמור אותו ל csv ונראה comments אל מול ה score

comments	sentiment_scaled	sentiment
My daughter and I had a wonderful stay with Maura. She kept in close touch with us throughout the day as we weren't	0.147274796	0.381168308
We stay at Elizabeth's place for 3 nights in October 2014.	0.379418829	0.425095259
If you're staying in South Boston, this is a terrific place to camp out. The apartment and bedroom are lovely, Ellie is an	-0.58917438	0.24181531
Derian and Brian were great and prompt with their communications with us. The room was as described; it was a sma	-0.696931618	0.221425181

לפעמים סטנדרטיזציה תיקח ביקורת חיובית ותוריד אותה מתחת ל0 למספרים מאוד קטנים.

```
write.csv(bos.airbnb,"bos.airbnb with polarity - compare scale and sentimentr.csv")
```

יש פונקציה שמראה את הטקסט בצורה יפה, מזהה משפטים שזוהו כשליליים או חיוביים

```
library(magrittr)
```

```
library(dplyr)
```

```
library(data.table)
```

```
bos.airbnb.table <- as.data.table(bos.airbnb)
```

נהפוך את הטקסט לטבלה. האופרטור %<% לוקח תוצאות מ bos.airbnb ומעביר אותן לחלק השני.

```
set.seed(1)
```

```
bos.airbnb.table %>% select(review_id,comments) %>%
```

כאן נבקש להפוך את התגובות למשפטים בודדים ותחשב עליהם סנטימנט עם פונקציית היי לייט.

```
filter(review_id %in% sample(unique(review_id), 50)) %>% #choose 50 reviews
```

```
mutate(review = get_sentences(comments)) %>%
```

15092566: +.238

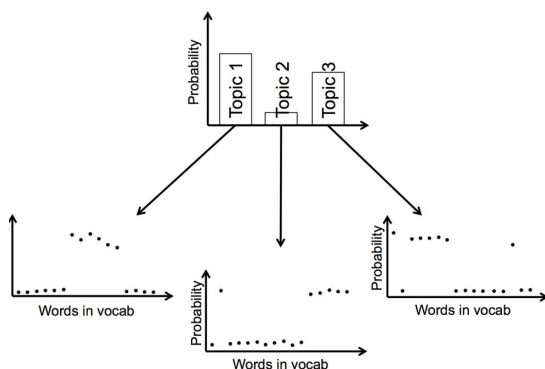
My son and I spent a week in Kevin's apartment. We enjoyed the neighborhood... it was far more relaxing than North End, Back Bay, etc. The apartment itself was spacious and quiet. Bonuses were the great kitchen and backyard. There were a couple of issues that Kevin and his assistant, Liesl, were very responsive to. Communications with them was always prompt and entry in and out of the apartment is so convenient.

Topic Modeling

לוקחים נתונים (לדוג' גובה ומשקל) ומחלקים אותם לאשכולות כדי למצוא קבוצות שבאותו אשכול הם מתנהגים בצורה

דומה. כן המטרה היא לחלק את המסמכים- למסמכים שהם מדברים על אותו נושא. למשל במערכות המלצה, יש אוסף גדול של מסמכים ורוצים להמליץ על הכתבה הבאה בווינט. אז אפשר מראש למצוא מודל שאם קראתם כתבה X נציע לכם כתבה עם תוכן שונה. Topic Modeling יכול לשמש גם כמשימה מקדימה למודלים של קלסיפיקציה.

המטרה שלנו היא לקחת אוסף של מסמכים ולהגיד האם המסמך שייך נושא 1 או נושא 2. מסמך יכול להשתייך גם לכמה נושאים. כל מסמך למעשה יכול להשתייך ב 80% לפוליטיקה ו 20% מזג אוויר. כל נושא יכול להיות מורכב ממילים שמאפיינות אותו.



יש כמה אלגוריתמים למציאה של נושאים, אנחנו נדבר על אלגוריתם LDA. הנושא הוא מורכב- הוא מוצא הסתברויות האלגוריתם בונה את ההתפלגות אבל יש לו מספר התפלגויות. הוא מוצא מה ההסתברות שמסמך מסוים שייך לנושא מסוים. כלומר רואים את ההתפלגות של הנושאים. ומצד שני הוא מוצא את ההתפלגויות של המילים בכל נושא.

Topic 1 (R code)	Topic 2 (Math terms)	Topic 3 (Case study)
library	number	HR
skmeans	cluster	work
dtm	ensemble	recruiting
corpus	K-Means	team
weight	Spherical	experience
cluster	weight	educate

Fuzzy assignment - words

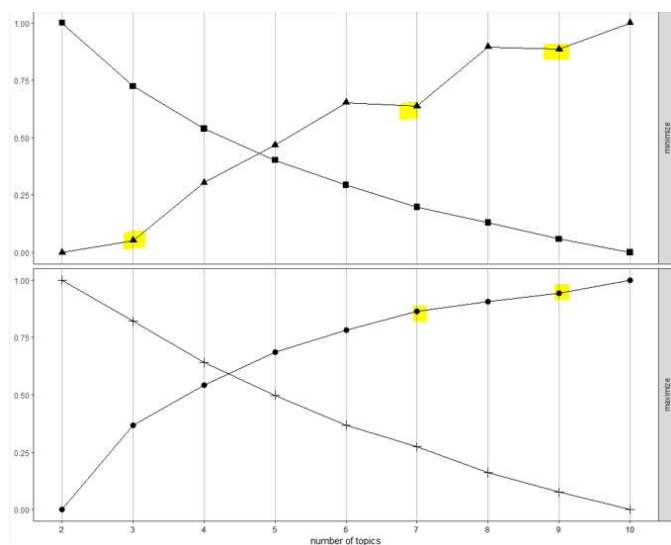
דוגמא לחלוקה של מילים במסמך (מסמך בו יש מילים מ 3 נושאים שונים)- מגדיר את המילים הכי בולטות בכל נושא. ניתן לראות שיש מילים שמופיעות בכמה נושאים לדוגמא המילה cluster- זה נקרא fuzzy assignment.

Document	Topic 1 (R code)	Topic 2 (Math terms)	Topic 3 (Case study)
1	0.31	0.29	0.39
2	0.34	0.23	0.42
3	0.31	0.25	0.43
4	0.35	0.36	0.28
5	0.28	0.45	0.25
6	0.34	0.31	0.36

Fuzzy assignment - topics

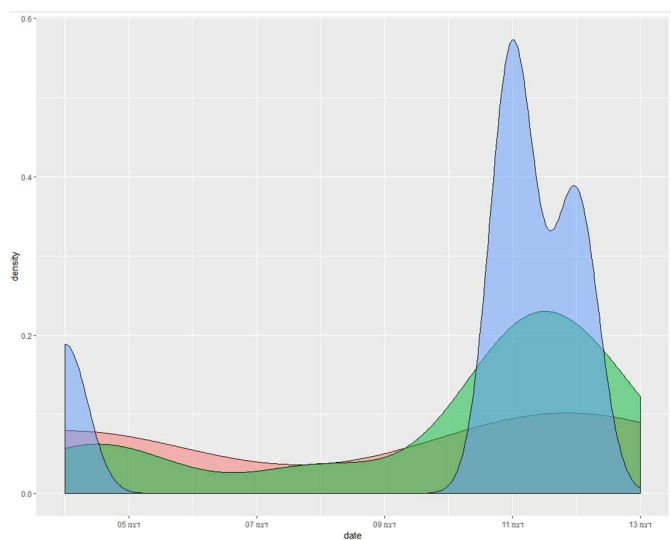
עבור כל מסמך נקבל את ההסתברות שלו להיות שייך לנושא מסוים- כלומר ניתן לומר שמסמך מספר 2 מורכב מ 2 נושאים, אבל בהסתברות הכי גבוהה הוא מתאים לנושא 3. החלוקה לאחוזים פה נראית יחסית מאוזנת, כלומר זה בדיוק פרמטר שנוכל לשלוט בו ולהגיד לו האם נרצה שהחלוקה תהיה יחסית מאוזנת בין הנושאים או שיתמקד וייתן משקל גדול יותר לנושא מסוים- וזה צריך להסתכם ל 100%.

אחד הדברים המורכבים הוא- מה כמות הנושאים שיש לחלק אליהם?

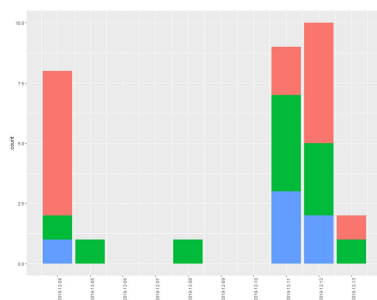


הכל תלוי במשימה וכמות הנתונים- אם יש אלפי מסמכים או מספר קטן. ומספר הקבוצות שנרצה להתעסק איתן במערכת ההמלצות שלנו.

יש ספרייה ב R - shdatuning. שנותנת 4 מדדים שמייצרים גרף. ציר X הוא מספר הנושאים (פה ניתן לראות 10) ותחשב את המדדים השונים. איך נדע מה המדד ממליץ? יש 2 מדדים שמגדירים מינימום ומקסימום- זה מה שהמדד ממליץ. בחלק העליון לדוגמא נרצה למצוא מינימום מקומי (הנקודות הצהובות). ואם נחפש מקסימום מקומי אז בגרף התחתון נראה ב 7 ו 9 (אפשר לדעת לפי מה שהגרף מגדיר- כאן ה μ זה $\max$ כתובים באפור בגרף).



אפשר להסתכל על הנושאים לאורך זמן- כאם למשל רואים tweets מחולקים לנושאים- וניתן לראות על איזה נושא דיברו בכל יום. לדוגמא ימים מסויימים שדיברו יותר על נושא הכחול.



הביקורת על השיטה-

כל פעם שמריצים אפשר לקבל תוצאות שונות- אין אמת ויש חוסר יציבות. אין משהו מאוד מדויק.

```
setwd("E:/Text mining course/4")
```

ניתוח טוויטים עם האשטג של ישראל. (49 טוויטים- ניתן לדעת מי העלה, מיקום שהועלה וכו- מה שחשוב זה הטקסט)

טעינת ספריות רלוונטיות:

```
library(topicmodels)
library(tm)
library(rtweet)
library(wordcloud)
library(textstem)
options(stringsAsFactors = F)
Sys.setlocale('LC_ALL','C')
```

קריאה של הקובץ ל df

```
tweets <- read.csv("israelTweetsPop25052022.csv")
```

```
class(tweets)
head(tweets$text, 5)
```

```
tweets$text <- gsub("http\\S*", " ", tweets$text)
tweets$text <- gsub("\n", " ", tweets$text)
tweets$text <- gsub("&", " ", tweets$text)
```

מכילים רשימה של stopwords- להוריד מילים שהם פופולאריות.

```
custom.stopwords <- c(stopwords("english"),'rt','israel','israels','amp','israeli','israelis')
```

ניקוי של הטקסט

```
clean.data <- function(text){
  myCorpus <- VCorpus(VectorSource(text))
  myCorpus <- tm_map(myCorpus, content_transformer(tolower))# convert to lower case
  myCorpus <- tm_map(myCorpus, removeWords, custom.stopwords) #remove stopwords
  myCorpus <- tm_map(myCorpus, removeNumbers)# remove numbers
  myCorpus <- tm_map(myCorpus, removePunctuation)# remove punctuation
  myCorpus <- tm_map(myCorpus, stripWhitespace)# remove extra whitespace
```

לא חייבים לעשות stemDocument אם זה מקשה על הבנת המילים.

```
myCorpus <- tm_map(myCorpus, stemDocument)
return(myCorpus)
}
tweetCorpus <- clean.data(tweets$text)
```

מדפיסים 5 שורות כדי לראות אם הטוויטים נראים אחרי ניקיון

```
for(i in 1:5){  
  print(tweetCorpus[[i]][1])  
}
```

לפני שמריצים topic modeling צריך לייצר DocumentTermMatrix:

כלומר כל תא ב dtm יכול כמה פעמים כלמילה מופיעה במסמך מסוים. עוד שברים שנוהגים לעשות, זה לצמצם מילים שמופיעות מעט מאוד פעמים. למשל כשנרצה להשאיר רק מילים שמופיעות יותר מפעם 1.

```
dtm <- DocumentTermMatrix(tweetCorpus)
```

דבר ראשון שנרצה, זה להבין לכמה נושאים לחלק, נוכל לשחק עם זה ידנית או בהתחלה לראות מה המדדים ממליצים לי, נשתמש עבור זה בספריית ldatuning.

```
library(ldatuning)
```

```
result <- FindTopicsNumber(  
  dtm, topics = seq(from = 1, to = 10, by = 1),  
  metrics = c("Griffiths2004", "CaoJuan2009", "Arun2010", "Deveaud2014"),  
  method = "Gibbs",  
  control = list(seed = 10, alpha = 0.1, mc.cores = 2L, verbose = TRUE)  
)  
FindTopicsNumber_plot(result)
```

נרץ את המפונקציה הבאה (הרצה של פונקציות בין 1 ל - 10)

מגדירים אילו מדדים נרצה

```
metrics = c("Griffiths2004", "CaoJuan2009", "Arun2010", "Deveaud2014"),
```

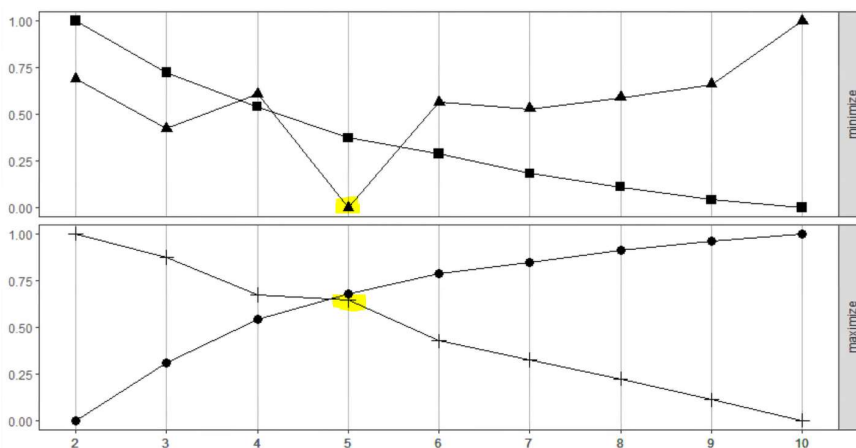
הגדרת המתודות יש VEM/Gibbs (שיטת VEM יותר ישנה)

```
method = "Gibbs",
```

נרצה לקבע את ה seed בשביל שהתוצאות לא יצאו שונות כל פעם, alpha אם נרצה לשחק עם מידת ה fuzziness- שמסמך מסויים ישתייך בסבירות גבוהה לנושא מסוים.

```
control = list(seed = 10, alpha = 0.1, mc.cores = 2L, verbose = TRUE)
```

```
FindTopicsNumber_plot(result)
```



נקבל תוצאה של המדדים-

נראה שהכי טוב יהיה 5

נריץ את האלגוריתם LDA על ה DocumentTermMatrix עם 5 נושאים. שאר המדדים זהים למה שהגדרנו בפונקציה הקודמת כדי שיהיה זהה. שינוי alpha ישנה את הטווחים ב posterior. ככל שה alpha יותר גדולה = יותר נושאים.

```

Topic 1      Topic 2      Topic 3      Topic 4      Topic 5
[1,] "palestin" "will"      "state"      "palestinian" "minist"
[2,] "can"      "militari" "antisemit" "today"      "ukrain"
[3,] "mission"  "realli"   "jew"        "kill"        "report"
[4,] "peopl"    "day"      "know"       "ambassador"  "energi"
[5,] "countri"  "hate"     "one"        "big"         "good"
[6,] "law"      "iran"     "call"       "close"       "increas"
[7,] "man"      "say"      "citi"       "fault"       "secur"
[8,] "may"      "southern" "member"     "flag"        "common"
[9,] "rememb"   "target"   "toward"     "hear"        "cooper"
[10,] "around"  "truth"    "antizion"   "like"        "crucial"

```

```

lda <- LDA(dtm, k = 5,
control=list(seed=10,alpha=0.1),method = "Gibbs")
#ניח שנרצה לראות מה המילים שמאפיינות כל נושא (עם 10 המילים הראשונות)
termsByTopic <- terms(lda, 10)
termsByTopic

```

נוכל לראות את ההסתברויות אם נפנה לפונקציית posterior של כל מסמך ואיך החלוקה של כל מסמך לכל נושא. לדוג בהסתברות של 62% המסמך הראשון שייך לנושא 1. או את ההסתברות של כל מילה להיות במסמך מסוים.

```

posterior(lda)$topics[1:10,] #topic distribution
for each document (ten first documents)
posterior(lda)$terms[,1:10] # term distribution
among topics
head(tweets$text, 4)

```

	1	2	3	4	5
1	0.620512821	0.005128205	0.364102564	0.005128205	0.005128205
2	0.514893617	0.472340426	0.004255319	0.004255319	0.004255319
3	0.221052632	0.010526316	0.010526316	0.010526316	0.747368421
4	0.131914894	0.004255319	0.812765957	0.004255319	0.046808511
5	0.387234043	0.004255319	0.131914894	0.387234043	0.089361702
6	0.127272727	0.066666667	0.006060606	0.248484848	0.551515152
7	0.004081633	0.004081633	0.983673469	0.004081633	0.004081633
8	0.088000000	0.008000000	0.008000000	0.888000000	0.008000000

	aahronheim	absolut	abu	account	accumul	accus	acquir	activist	actual
1	0.0004389816	0.0048287972	0.0048287972	0.0048287972	0.0004389816	0.0048287972	0.0004389816	0.0004389816	0.0048287972
2	0.0042834891	0.0042834891	0.0003894081	0.0003894081	0.0003894081	0.0003894081	0.0042834891	0.0003894081	0.0003894081
3	0.0004135649	0.0004135649	0.0004135649	0.0004135649	0.0004135649	0.0004135649	0.0004135649	0.0004135649	0.0004135649
4	0.0004277160	0.0004277160	0.0004277160	0.0004277160	0.0004277160	0.0004277160	0.0004277160	0.0047048760	0.0004277160
5	0.0004721435	0.0004721435	0.0004721435	0.0004721435	0.0051935788	0.0004721435	0.0004721435	0.0004721435	0.0004721435

נרצה להריץ את הפונקציה topics על מודל ה lda ונוכל לראות לאיזה נושא כל מסמך השתייך. יש 49 מסמכים ולדוגמא המסמך ה 1 השתייך לנושא 1.

```

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42
1 1 5 3 1 5 3 4 4 4 2 4 2 4 2 2 3 1 5 5 5 4 1 5 2 2 5 3 1 3 4 1 2 2 5 1 3 1 3 2 4 4
43 44 45 46 47 48 49
3 2 5 4 4 3 2
I

```

```
topics <- topics(lda)
```

כאן נוכל לראות בכל נושא כמה מסמכים יש

```

topics
1 2 3 4 5
table(topics)
9 11 9 11 9

```

נוכל לתת שמות לנושאים :

```
library(dplyr)
```

```
topics.labeled <- recode(topics, '1'='Topic 1','2'='Topic 2','3'='Topic 3', '4'='Topic 4', '5'='Topic 5')
```

```
table(topics.labeled)
```

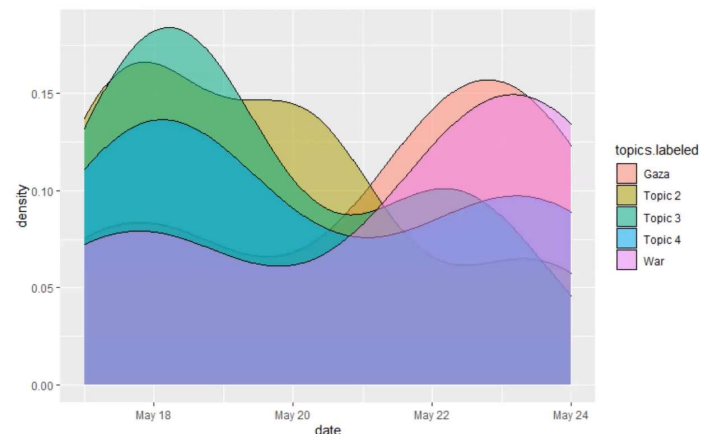
נרצה ליצור את הגרפים עם התאריכים: נבנה dataframe חדש שמכיל עמודת תאריך (שנוצר ה tweet) ונושא ה tweet.

```
topics.by.date <- data.frame(date=as.Date(tweets$created_at),
topic=factor(topics.labeled))
library(ggplot2)
```

	date	topic
1	2022-05-24	Gaza
2	2022-05-18	Gaza
3	2022-05-22	War
4	2022-05-23	Topic 3
5	2022-05-23	Gaza
6	2022-05-17	War
7	2022-05-22	Topic 3
8	2022-05-18	Topic 4

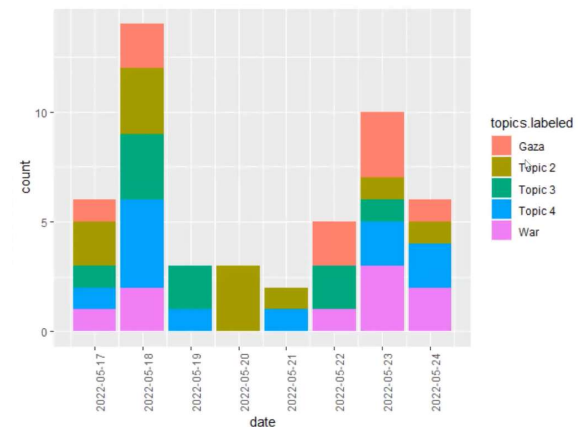
density plot:

```
ggplot(topics.by.date, aes(date, fill = topics.labeled)) +
geom_density(alpha = 0.5) # alpha is a transparency
param
```



bar plot:

```
ggplot(topics.by.date, aes(date, fill = topics.labeled)) +
geom_bar(position = "stack") +
scale_x_date(date_breaks = "days") + theme(axis.text.x
= element_text(angle = 90, hjust = 1))
```



נרצה להשתכל נושא- נושא ולחקור את זה יותר לעומק ואם יש היגיון בחלוקה- נייצר אקסל שמכיל את tweet ואת הנושאים אליו זה השתייך.

```
library(qdap)
```

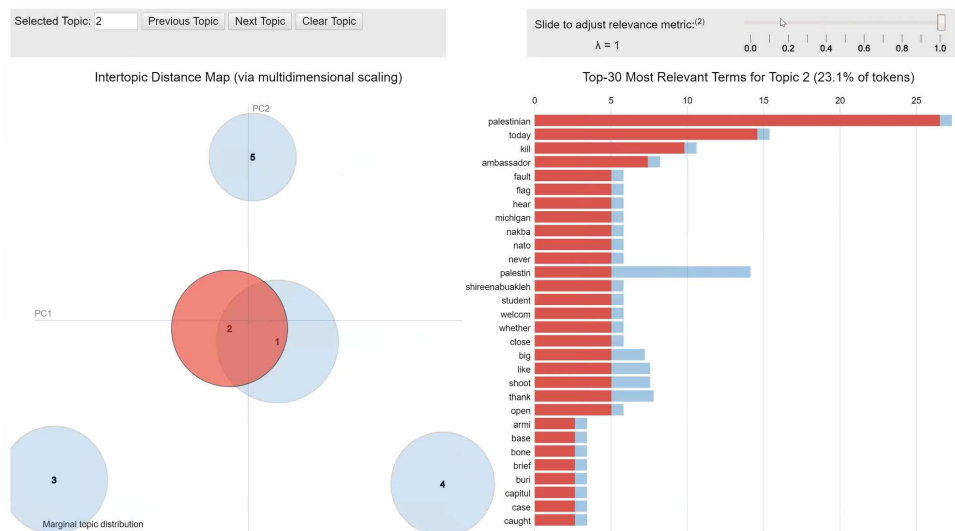
```
cleaned.df <- as.data.frame(tweetCorpus)
```

נבנה df חדש עם עמודת test המקורית עם התאריך בו נוצר, הנושא אליו האלגוריתם שייך את ה tweet ולראות את הטקסט (כדי להשוות עם איך שהוא נוקא)

```
tweets.with.topic <- data.frame(text = tweets$text, created = as.character(tweets$created_at),
topic = as.character(topics.by.date$topic),cleand_text = cleaned.df$text)
write.csv(tweets.with.topic, 'tweets_with_topic_pop_25052022.csv')
```

פונקצית topicmodels2LDAvis עוזרת לתחקר את התוצאות שמתקבלות (קוד מסובך ולא צריך להכנס אליו יותר מידי)
library(LDAvis)

```
topicmodels2LDAvis <- function(x, ...){ post <- topicmodels::posterior(x)
if (ncol(post[["topics"]]) < 3) stop("The model must contain > 2 topics")
mat <- x@wordassignments
LDAvis::createJSON(
  phi = post[["terms"]],
  theta = post[["topics"]],
  vocab = colnames(post[["terms"]]),
  doc.length = slam::row_sums(mat, na.rm = TRUE),
  term.frequency = slam::col_sums(mat, na.rm = TRUE))}
```



הרצת הפונקציה עם המודל שבנינו.

```
Jason <-
topicmodels2LDAvis(lda)
serVis(Jason)
```

מקטין את המימדים ומצייר אותה על
2 מימדים ומראה את המרחק בין
הנושאים. הקרבה בין 1 ל 2- אומר
שהם קרובים במילים שלהם.

יש אפשרות לבנות את המודל שיועד לשייך מסמך חדש שנוסף לתוך המודל הקיים ששומר בתוכו את המאפיינים של כל
נושא- יש ds של train ושל validation. לוקח מספר מסמכים אחרונים ומריצים lda רק על ה train ואז נוצר מודל- אם
מגיעים מסמכים חדשים נוכל להריץ את הפונקציה של posterior ולתת לו את המודל והמסמכים החדשים.
התוצאה שלו זה שעל כל מסמך חדש- נראה את ההסתברות של שיוך לנושא.

```
dtm.train <- dtm[1:(dtm$nrow-5),]
dtm.valid <- dtm[(dtm$nrow-4):dtm$nrow,]
lda.train <- LDA(dtm.train, control = list(alpha = 0.1,seed=10), k = 5)
lda.inf <- posterior(lda.train, dtm.valid) #compute posterior probabilities for new data
lda.inf
```

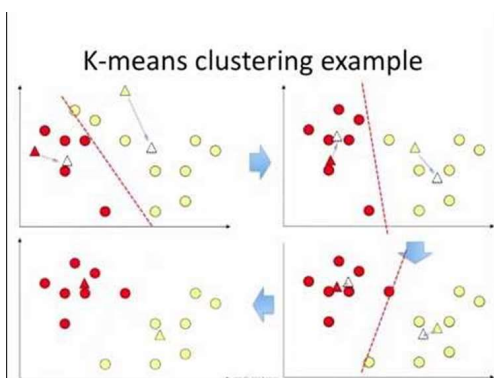
השיעור ידבר על שיטה של חלוקה לקבוצות - cluster analysis. המטרה היא לחלק את המסמכים לקבוצות על בסיס דמיון בין מסמכים. משתמשים בזה למשל באוסף של כתבות ורוצים להמליץ על כתבות דומות אז נחלק את אוסף הכתבות לקבוצות וברגע שנסתכל על כתבה מסויימת נוכל להציע כתבות דומות.

	Term 1	Term 2	Term 3	Term 4	...	Term p
Document 1						
Document 2						
Document 3						
Document 4						
Document 5						
...						
Document n						

האלגוריתם של cluster analysis - בונה מטריצה ובודק עבור כל מסמך אילו מונחים יש בו. ומחפש עוד מסמך עם מונחים דומים. כך שאם יש 2 מסמכים שמכילים את אותם המונחים - הם יהיו תחת אותה הקטגוריה. הרעיון הוא לחשב מרחקים בין מסמכים. אם המרחק הוא קטן זה אומר שהמסמכים קרובים בתוכן.

האלגוריתמים שמבצעים זאת:

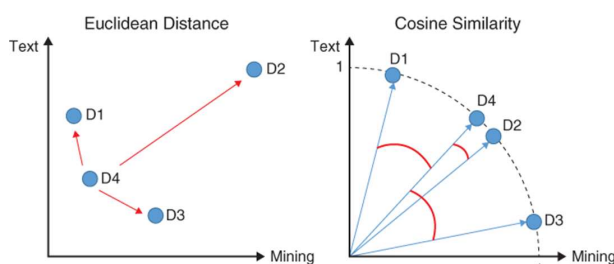
K-Means clustering



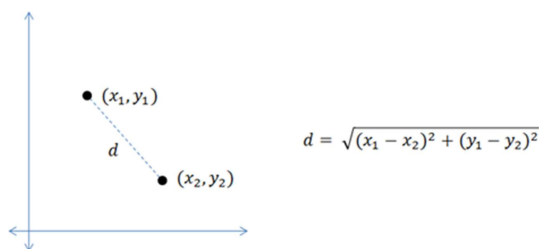
- אלגוריתם הכי פופולארי לניתוח אשכולות. האלגוריתם לוקח אוסף של מסמכים ומחפש את הדמיון בניהם ומייצר clusters.
 - איך עובד האלגוריתם? הנקודות הן מסמכים.
 1. בחרים 2 נק' באופן רנדומלי.
 2. כעת נעבור מסמך מסמך ונבדוק למי המסמך יותר קרוב? לנקודה ה-1 או ה-2 שנבחרה? לפי זה נשייך אותו לאשכול.
 3. מחשבים את הממוצע של כל הצהובים ומעדכנת את המרכז. לדוגמה המשולש הלבן.
 4. עושים את אותה הפעולה שוב (צעד 2 ו 3) - כאשר מסיימים לשפר את ההומוגניות בכל אשכול מפסיקים.
- כעת ישנם 2 אשכולות של מסמכים שדומים אחד לשני.
- החסרון: צריך להחליט על מספר הקבוצות מראש - יש שיטות להחליט על מס' קבוצות.

כיצד ניתן לעבוד עם האלגוריתם כאשר מדובר בטקסטים?

המימדים יהיו דומים למה שאנחנו מכירים - document matrix, עבור כל מילה כמה פעמים היא מופיעה במסמך. בעצם, המימדים פה הם אורך של כל המילים בכל corpus.

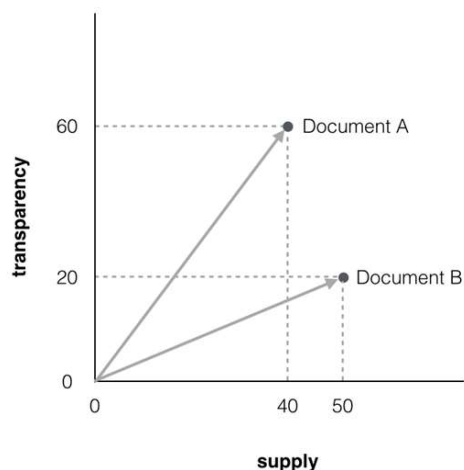


אחרי שבנינו את ה document matrix, נבדוק האם 2 מסמכים דומים אחד לשני. יש 2 גישות למדידת מרחקים: מרחק אוקלידי ומרחק לפי זווית (Cosine similarity). $\cos(90)=0$ יהיה המרחק הכי גדול בין 2 מסמכים - ככל שהם יותר רחוקים הדימיון הוא 0.



Euclidean Distance - לפי נקודות X ו Y . לפי משט פתגורס. אם הגובה והמשקל הם מאוד שונים (לדוג' קילוגרמים וסנטימנטים) אז נבצע נרמול. בהקשר של documents נהיה צריכים גם לבצע נירמול. בהקשר של tf-idf. ה idf כבר מנרמל את הנתונים כי הם כבר מנורמלים.

Cosine Similarity - חישוב סמיון בין 2 מסמכים-



$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}},$$

$$\cos(\theta) = \frac{40 \times 50 + 60 \times 20}{\sqrt{40^2 + 60^2} \times \sqrt{50^2 + 20^2}} = 0.82$$

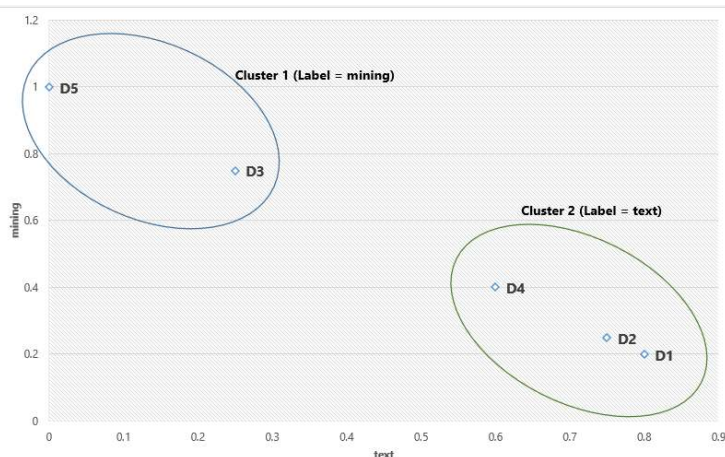
ההצבה במסמך בהתאם לשרטוט.

דוגמא על מסמכים:

Document	Text
D1	Text <u>text</u> <u>text</u> <u>text</u> mining
D2	Text <u>text</u> <u>text</u> mining
D3	Text mining <u>mining</u> <u>mining</u>
D4	Text <u>text</u> <u>text</u> mining <u>mining</u>
D5	Mining <u>mining</u> <u>mining</u>

Document	Term = Text	Term = Mining
D1	4/5=0.8	1/5=0.2
D2	0.75	0.25
D3	0.25	0.75
D4	0.6	0.4
D5	0	1

- ישנם 5 מסמכים: אם נרצה לחשב את הדמיון.
- נבנה את ה dtm. הוא יהיה מצומצם כי יש ביטוי לטקסט ול mining.
- נבדוק כמה פעמים כל מונח מופיע. (למסמך הראשון זה 4 פעמים מתוך 5). נבדוק זאת גם לגבי mining.
- נצייר את המספרים על הגרף- בגלל שיש 2 פיצ'רים נוכל לצייר את זה עם גרף. וכך נחלק לקבוצות.

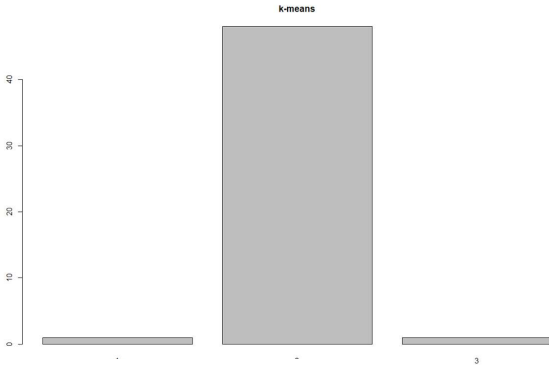


דוגמא לcluster analysis לקורות חיים עבור HR:

המטרה: לזהות את התכונות שמאפיינות נציגי שירות שהם טובים ונשארים לאורך זמן.

דרכי פעולה:

1. ניקח קו"ח ממקום העבודה האחרון.
2. נארגן אותם כדי שהאלגוריתם יכול לנתח אותם על ידי document metrix לפי משקל TF IDF לתת יותר משקל למילים חשובות.
3. נריץ את k means.
4. ננתח את התוצאות ונבין מה המאפיינים של כל איזור
5. הסקת מסקנות- לנסות להגיע לתובנות ולהבין האם זה תורם לי.



אחרי הרצת ה **K Means** - נקבל חלוקה לא מאוזנת בין האשכולות.

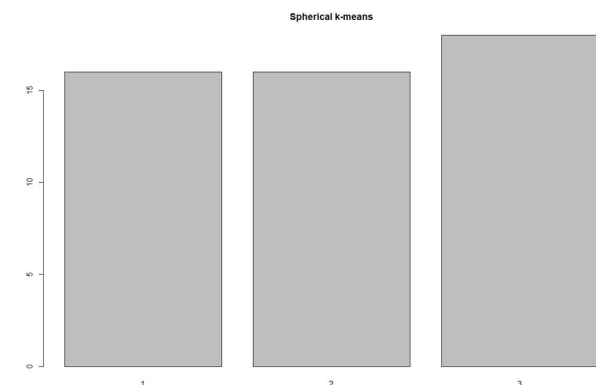
נריך comperison cloud בשביל לראות את ההבדל בין המילים. נוכל לראות דרך המילים לדוגמא שאם גייסנו מחברה מסוימת - האנשים שהגיעו משם באמת יותר טובים.



Findings Spherical K-Means ננסה להריץ גם עבור

משתמש ב cosine similarity. פה כבר נראה חלוקה יותר מאוזנת.
אחנו רואים שה 50 קו"ח מתחלקים יחסית שווה בשווה בין האשכולות.

בהרצת cloud comperison נקבל את ההבדלים בין ה clusters. המילים לא בדיוק ברורות בשביל להגיע לתובנות. לכן צריך לקחת את הקובץ פלט- עם עמודה שיסיביר לאיזה אשכול הוא משתייך ולנסות להבין את הקשר.



הביקורת על האלגוריתם-

הוא שהוא לא יציב. כל פעם שנריץ את זה נקבל תוצאות שונות. הוא מתחיל מ 2 נק' בצורה רנדומלית ולכן לא נקבל כל פעם את אותה התשובה.



דוגמא מהקוד:

הגדרות בסיסיות:

```
options(stringsAsFactors = F)
Sys.setlocale('LC_ALL','C')
```

```
library(skmeans)
library(tm)
library(wordcloud)
library(textstem)
library(stringr)
```

ניקוי של הטקסט:

```
clean.data <-function(text){
  corpus <- VCorpus(VectorSource(text))
  corpus <- tm_map(corpus, content_transformer(tolower))
  corpus <- tm_map(corpus, removeWords, c(stopwords('english'), "customer","service","customers","calls"))
  corpus <- tm_map(corpus, removePunctuation)
  corpus <- tm_map(corpus, removeNumbers)
  corpus <- tm_map(corpus, stripWhitespace)
  corpus <- tm_map(corpus, stemDocument)
  return(corpus)}
```

נקרא את הקובץ שנקרא Resumes.csv עם אוסף של קו"ח

```
resumeDf<-read.csv('Resumes.csv', header = T)
```

נעזר מהטקסט תוים מיוחדים שלא נרצה שיופיעו בטקסט.

```
resumeDf$text <- iconv(resumeDf$text, "latin1", "ASCII", sub=" ")
```

נוסיף עוד ניקיון לדוגמא עבור מילים צמודות עם פסיק - נחליף את הפסיק ברווח.

```
resumeDf$text <- str_replace_all(resumeDf$text, "[[:punct:]]", " ") #replace punctuation marks with space
```

נרץ את הפונקציה נקיון

```
resumeCorpus <- clean.data(resumeDf$text)
```

נסתכל על ה- 5 קו"ח הראשונים

```
for(i in 1:5){ print(resumeCorpus[[i]][1])}
```

אחרי שניקינו את את הקורפוס נבנה את ה dtm לפי tf-idf.

```
resumeDTM <- DocumentTermMatrix(resumeCorpus,control = list(weighting = weightTfIdf))
```

```
inspect(round(resumeDTM,4)[1:5,1:5])
```

ניתן להוריד גם מילים מאוד נדירות שמופיעות לדוג' פעם 1 או מילים מאוד נפוצות. (לא חייב להרץ את השורה הזאת)

```
resumeDTM <- removeSparseTerms(resumeDTM, .98) # 0.98 - the maximal allowed sparsity per term.
```

נרץ את ה K-Means

```
set.seed(1)
```

נחלק ל 3 אשכולות ואת התוצאה נשים בתוך אובייקט שנקרא resumeClusters

```
resumeClusters<-kmeans(resumeDTM,3)
```

```
resumeClusters$cluster
```

```
1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  1  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3
41 42 43 44 45 46 47 48 49 50
3  3  3  3  2  3  3  3  3  3
```

נראה שאין חלוקה מאוזנת- כמעט כל המסמכים באשכול מספר 3.

```
barplot(resumeClusters$size, main='k-means')
```



מריצים את ה spherical k means לפי 3 אשכולות. ניתן לראות את 2 הbarplots שיוצאים.

```
set.seed(1)
```

```
resumeClusters <- skmeans(resumeDTM, 3)
```

```
barplot(table(resumeClusters$cluster), main='Spherical k-means')
```

```
resumeClusters$cluster
```

נוסיף עמודה שנקראת cluster שתגיד לנו איפה המסמך מסווג.

```
resumeDf$cluster <- resumeClusters$cluster
```

כדי לייצר cloud comparison- נאחד את כל הטקסטים עבור כל אשכול

```
dataForAnalysis <- aggregate(resumeDf$text~resumeClusters$cluster, FUN = paste, collapse = " ")
```

ה DF מכיל 3 רשומות (עבור כל אשכול)- כל רשומה מכילה את כל המילים עבור אותו אשכול.

	resumeClusters\$cluster	resumeDf\$text
1	1	Mentor in training new hires Mentored other team member...
2	2	Assisting customers with their online orders via phone calls ...
3	3	Responsible for handling large cash amounts on a daily bas...

ננקח את הטקסטים על עמודה מספר 2 (הטקסט)

```
dataForAnalysisCorpus <- clean.data(dataForAnalysis[,2])
```

מייצרים מסמך של tf-idf

```
dataForAnalysisTdm <- TermDocumentMatrix(dataForAnalysisCorpus,control=list(weighting=weightTfIdf))
```

יש ליצור את זה כמטריצה

```
dataForAnalysisTdm.m <- as.matrix(dataForAnalysisTdm)
```

ניתן לראות את המילים והמסמכים- יש כעת מסמך עבור כל אשכול.

```
colnames(dataForAnalysisTdm.m) <- c("Cluster 1", "Cluster 2", "Cluster 3")
dataForAnalysisTdm.m[1:10,1:3]
```

Terms	Cluster 1	Cluster 2	Cluster 3
abl	0.002637209	0.000000000	0.000000000
accomplish	0.000000000	0.000000000	0.000000000
accord	0.000000000	0.001659468	0.000708187
account	0.000000000	0.000000000	0.000000000
accur	0.007911626	0.000000000	0.000000000
accuraci	0.000000000	0.000000000	0.001918841
achiev	0.000000000	0.000000000	0.001918841
act	0.000000000	0.000000000	0.001918841
action	0.000000000	0.000000000	0.001918841
activ	0.000000000	0.000000000	0.000000000

נריך comparison cloud על המטריצה. כעת יש לנו את ההבדלים בין המסמכים באשכולות.

```
comparison.cloud(dataForAnalysisTdm.m,
  max.words = 50,
  random.order = FALSE,
  colors = c('red','blue', 'green'), title.size=1)
```



נוכל גם לייצר טבלה שבודקת מה המילים עם המשקל הכי גבוה בכל אחד מהאוסף מסמכים.

```
data.frame(cluster1 =
  names(sort(dataForAnalysisTdm.m[,1],decreasing=T)[1:6]),
  cluster2 = names(sort(dataForAnalysisTdm.m[,2],decreasing=T)[1:6]),
  cluster3 = names(sort(dataForAnalysisTdm.m[,3],decreasing=T)[1:6]))
```

	cluster1	cluster2	cluster3
1	concis	help	manag
2	present	answer	creat
3	accur	credit	design
4	behalf	incom	freight
5	medicar	bilingu	media
6	multipl	card	social

לפי מה הוא מייין את המילים? לפי כמות הפעמים שהמילה הופיעה בכל הקו"ח שנאספו.

נוכל להוסיף עמודה של cluster לקובץ המקורי, לשמור אותו ולהסתכל עליו באקסל.

```
resumeDf$cluster <- resumeClusters$cluster
write.csv(resumeDf, "Resume with cluster 2021.csv")
```

נוכל להסתכל על הצנטרואידים (מרכזי ה clusters) ולבנות את comparison cloud בהתבסס על הצנטרואידים. נוכל לעשות שימוש בפונקציה cl_prototypes.

resumeClusters -נוצר בחלוקה לאשכולות ב k-means. נריך עליו פונקציה שנקראת cl_prototypes -שנותן את המרכזים עבור כל אשכול (זה סוג של וקטור ממוצע). למילים שיוצאות בפלט יש משקל גבוה אם הם מאפיינות את האשכול.

```
library(clue)
s.clus.proto <- t(cl_prototypes(resumeClusters))
```

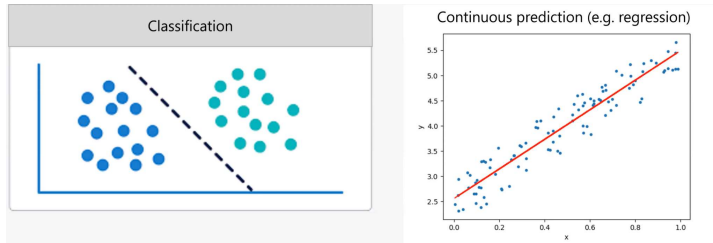
	1	2	3
abl	0.02556096	0.00000000	0.00000000
accomplish	0.02572491	0.14886449	0.006948873
accord	0.00000000	0.03142288	0.019945321
account	0.07697087	0.19384034	0.027162878
accur	0.07266556	0.00000000	0.000000000
accuraci	0.00000000	0.00000000	0.022343143
achiev	0.00000000	0.00000000	0.035215038
act	0.00000000	0.00000000	0.019048307

[illegible]

	cluster1	cluster2	cluster3
1	appropri	inbound	target
2	concis	skill	manag
3	place	account	client
4	issu	answer	product
5	order	take	perform
6	requir	use	depart
7	busi	assist	com
8	provid	provid	daili
9	store	order	shoe
10	experi	accomplish	media

Document Classification-

לדוגמא, זיהוי האם מסמך מסוים הוא spam על ידי ייצור אלגוריתם של קלסיפיקציה שמבין שבמקרה של צירוף מילים מסוים לדוגמא, זה spam.



שמדברים על Document Classification - שדה המטרה שלנו (המשתנה התלוי) הוא או בינארי או קטגוריאלי. לעומת בעיות רגרסיה, אשר חוזות כאשר משתנה המטרה הוא רציף (לדוגמא ציון במבחן).

Document ID	Text	Class
1	This is spam	yes
2	This is not spam	no
3	This is also spam	yes

Docs	also	not	spam	this	Class
1	0	0	1	1	yes
2	0	1	1	1	no
3	1	0	1	1	yes

Training data	Document ID	Text
	1	This is the first
	2	This is the second
Testing data	Document ID	Text
	1	This is testing data
	2	The model uses the original terms

מה מיוחד בסיווג טקסט? ואילו אלגוריתמים מתאימים לסיווג טקסט? כאשר מסווגים טקסטים, תהיה לנו עמודת טקסט ועמודה שמגדירה את הסיווג שלה (עמודת class).

נבנה document matrix, כל מילה תהיה פיצ'ר, נריץ אלגוריתם שנקרא cv.glmnet - סוג של רגרסיה.

הפונקציה תקבל את ה dtm ואת עמודת ה class שיבנה לנו מודל חיזוי.

בהינתן טקסט חדש, נכניס אותו למודל ונקבל פלט של התוצאה.

מה שמיוחד כאן, זה שהפיצ'רים שלנו חייבים להיות קבועים, ה dtm נבנה על בסיס מילים מסוימות. נרצה את אותו המבנה ונהיה צריכים את אותו אוסף של מילים. ב testing יש מילים כמו model, original, terms - כל אלה לא קיימים ב training. ולכן ניתן להשתמש רק במילים שיש ב training כדי לחזות על נתונים חדשים.

ניצור מה training את ה dtm ונגדיר לו לקחת רק מילים משם. כלומר, חייבים מבנה זהה של dtm.

case study - clickbait

פוסטים של פרסומות בפייסבוק (כאשר יש כתבה או פוסט עם כותרת מאוד "אטרקטיבית" אבל היא מטעה בתוכן שלה). פייסבוק ניסתה למגר את התופעה וחיפשו אלגוריתם סיווג שמזהה האם זה clickbaite או לא.

מה התהליך לזיהוי:

1. הגדרת המטרה - זיהוי clickbaite.
2. איסוף דאטה מתויג (שיש לנו תשובה לגבי ה class שלהם)
3. ארגון וניקוי הטקסט - הפרדה ל training ו testing - כלומר חלק מהנתונים יהיו למידה וחלק הערכת המודל.
4. בניית פיצ'רים על ידי dtm במשקל tf-idf לנרמול.
5. בניית המודל על ידי glm (רגרסיה מתקדמת) כדי לחשב מדדי הערכה.
6. פירוד התוצאות - והוצאת המילים עם הכי הרבה משמעות.

מה הם מדדי ההערכה למודל-

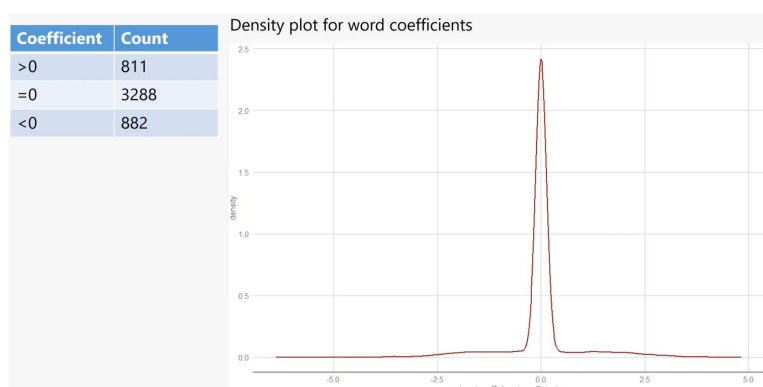
AUC- משקלל גם sensitivity וגם specificity- הכי טוב קרוב ל 0
Confusion Matrix- מטריצה לחיזוי. הכחול- החיזוי שצדקתי בו (1390) והאדום חיזוי שהוא טעות (110).

		Actual classes	
		0 = legitimate headlines	1 = clickbait
Predicted classes	0 = legitimate headlines	723	83
	1 = clickbait	27	667

מדד ה Accuracy- כמה פעמים צדקתי מבין סך כל הנתונים (0.926).

```
test.dtm <- DocumentTermMatrix(clean.test, control = list(dictionary=Terms(train.dtm),
weighting = weightTfIdf))
test.matrix <- as.matrix(test.dtm)
test.matrix <- Matrix(test.matrix)
#predict on the test data
preds <- predict(cv, test.matrix, type='class', s=cv$lambda.min)
```

מבחינת הקוד- על נתוני ה test נרצה לבנות באתו מבנה כמו ב train. ולכן שנבנה את ה dtm נרשום שהמילון שווה ל terms של ה train. שנרצה לבצע את החיזוי ברשומות החדשות נשתמש ב predict- ניתן לה את המודל ואת הנתונים. על ה test בד"כ נקבל תוצאות פחות טובות.



נרצה לראות בכמה מילים הוא השתמש כדי להחליט האם כותרת מסוימת היא clickbait או לא.
מה שנראה במודל רגרסיה- כאשר יש אוסף גדול של הרבה מימדים כל מילה תקבל מקדם שיגדיר כמה היא משפיעה (מקדם גדול יותר).
לדוג' במייל של spam יש הרבה מילים שחוזרות על עצמן. מילים שחוזרות על עצמן יקבלו מקדם גבוה.
ה count מגדיר את כמות המילים שהיו במודל- 3288 מילים לא משפיעות על המודל. ל 811 יש השפעה חיובית ול 882 יש השפעה שלילית.

דרכים לשיפור המודל-

1. **feature engineering**- בד"כ בהרצה ראשונה של מודל סיווג יש לשחק עם הפרמטרים, לשפר ניקיון, לנרמל פיצ'רים ועוד.
2. **Tokenization**- במקום unigrams להוסיף לדוגמא bi-grams (כמות המילים שתכנס - חלוקה לפי קבוצות של כמות המילים שתכנס).
3. **alpha penalty**- פרמטר שמגדיר עד כמה המודל גנרי. ככל שהalpha מתקרה ל 1 הוא יותר גנרי.
4. **prediction stacking and ensemble modeling**- חיבור מספר מודלים כאשר בוחרים את מה שרוב המודלים נתנו.
5. הוספת דאטה למודל.

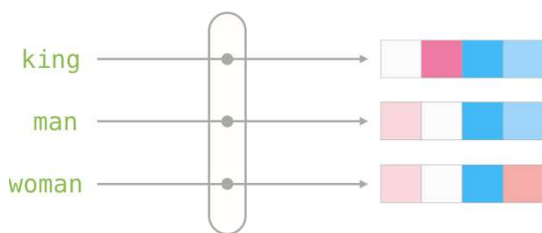
בניית אלגוריתם חיזוי- עבור חולה מאושפז- מה סיכוי שיחזור לבית החולים?
 חיזוי מסוג Readmission- נותן אפשרות לבית החולים להיערך ולכן חשוב.
 עבור חולה מסוים אם יש לי גיל, מגדר, טקסט שהרופאים כתבו ואני רוצה לשלב אותם- צריך להבין כיצד עושים זאת.
 עד היום השתמשנו בשיטת Bag of words (בניית dtm שיש לכל מילה ועמודה- רושמים במטריצה האם קיימת או לא קיימת)

אנו יודעים שחסרונות השיטה הם:

1. ניתן לבדוק רק את המילים הקיימות, אם נרצה לבדוק מסמך חדש, נבדוק רק את המילים שהיו במטריצה.
2. לא מתייחס ל context- אם המילה ike קיימת והיא מקושרת לסנטימנט חיובי, בפעם הבאה נדע לקשר אותה לסנטימנט חיובי. אם תהיה מילה נוספת הנרדפת ל like המודל לא יכיר אותה.
3. אין משמעות לסדר ב bag of words. הפתרון לזה הוא שימוש ב b-grams.

בגלל החסרונות הנ"ל נמצא מודל word2vec. המודל הינו הבסיס למודלים מתקדמים ל deep learning. הבסיס עובד טוב ככל שיש לו יותר מילים.

Word2vec



הרעיון הוא שבמקום לבנות dtm נמדל כל מילה כוקטור. למשל המילה king תקבל ייצוג וקטורי באורך 4. אם יש בין מילים דמיון אז הוקטור דומה. לדומא כאן man ו woman הוקטור דומה ולכן הצבעים זהים ב 3 מתוך 4. יש משמעות לצבע ולסדר.

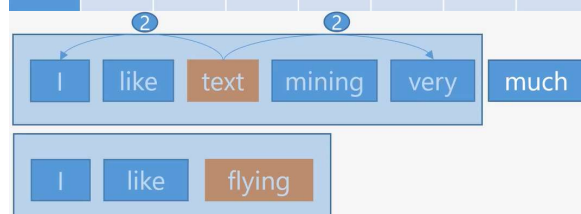
איך מייצגים את הוקטור למילה?

לוקחים הרבה מסמכים, בודקים מילים סמוכות לאותה המילה ועל ידי חישוב מייצרים וקטור. החישוב של הוקטור מיוצג על חישוב נקרא co-occurrence metrix- לפני שמייצרים וקטורים מייצרים מטריצה.

	I	like	text	mining	very	much	flying
I			1				1
like			1				1
text	1	1	0	1	1	0	0
mining			1				0
very			1				0
much			0				0
flying	1	1	0	0	0	0	0

לוקחים מילה מסוימת ובודקים איזה מילים נמצאות לידה ובודקים לדוגמא על המילה text- מופיעה ביחד I ו like וככה סופרים וממלאים את המטריצה. נקרא skip gram window כאן המספר הוא 2 (מימין ושמאל).

בנוסף, בודקים האם המילים שקרובות יותר למילה צריכות לקבל משקל גבוה יותר (לדוגמא ל mining נותנים משקל גדול יותר מ very) מה שקורה בפועל זה שהוא יקבל ציון גבוה יותר.



"Lion is the king of the jungle."



"The tiger hunts in this forest."

"Everybody loves New York."

נרצה לעשות text2vec - נרצה לדעת האם טקסט דומה לטקסט אחר.
נחשב ממוצע של המילים במשפט.
וקטור המייצג את כל המסמך (שורה) ונוכל לראות קרבה בין משפטים
(האדום והכחול דומים יותר).

תהליכי בניית המודל-

1. הגדרת מטרה- האם חולה יחזור לבית חולים תוך 30 ימים.
2. איסוף מידע- רשימות של 8500 חולי סכרת הכוללים את סיכום הביקור שלהם בזמן האשפוז.
3. ארגון המידע- נחלק את הנתונים ל train ו test.
4. בניית פיצ'רים- נייצג את המידע לפי bag of words ו text2vec.
5. נשתמש ב- 2 אלגוריתמים שונים Lasso ו רגרסיה מסוג SVM.
6. בדיקת תוצאות- בחירת המודל הטוב ביותר בתוצאות.

הקוד בR:

התקנת הסיפריות הרלוונטיות:

```
library(tm)
library(text2vec) # text2vec representation
library(glmnet) # lasso regression
library(caret) # confusionMatrix
library(e1071) # SVM
library(qdap) # corpus to data.frame
```

קריאת הנתונים: בסוף יש משתנה תלוי (האם אושפז כעבור 30 יום כן או לא)

```
options(stringsAsFactors = F) # make sure strings are read as strings, and not factors
diabetesDF <- read.csv("diabetes_subset_8500.csv")
```

איחוד הטקסטים לטקסט 1:

```
diabetesDF$diag.text <- paste(diabetesDF$diag_1_desc, diabetesDF$diag_2_desc,
diabetesDF$diag_3_desc, sep=' ')
```

	race.unknown	race.AfricanAmerican	race.Asian	race.Caucasian	race.Hispanic	race.Other	gender.Female	gender.Male	age.0.10	age.10.20	age.20.30
1	0	0	0	1	0	0	1	0	0	0	0
2	0	0	0	1	0	0	1	0	0	0	0
3	0	0	0	1	0	0	0	1	0	0	0
4	0	1	0	0	0	0	1	0	0	0	0
5	0	1	0	0	0	0	1	0	0	0	0
6	0	0	0	1	0	0	0	1	0	0	0
7	0	0	0	1	0	0	1	0	0	0	0
8	0	0	0	1	0	0	1	0	0	0	0
9	0	0	0	1	0	0	0	1	0	0	0
10	0	0	0	1	0	0	0	1	0	0	0
11	0	1	0	0	0	0	0	1	0	0	0
12	0	0	0	1	0	0	1	0	0	0	0
13	0	1	0	0	0	0	0	1	0	0	0
14	0	0	0	1	0	0	0	1	0	0	0
15	0	1	0	0	0	0	0	1	0	0	0

```
preprocess <- function(text){
  myCorpus <- VCorpus(VectorSource(text))
  myCorpus <- tm_map(myCorpus, content_transformer(tolower))
  myCorpus <- tm_map(myCorpus, removeWords, stopwords("english"))
  myCorpus <- tm_map(myCorpus, removeNumbers)
  myCorpus <- tm_map(myCorpus, removePunctuation)
  myCorpus <- tm_map(myCorpus, stripWhitespace)
  myCorpus <- tm_map(myCorpus, stemDocument)
  return(myCorpus)
}
```

מחלקים את הנתונים ל-60% train ו-40% test

```
set.seed(1)
trainIDs<-createDataPartition(diabetesDF$readmitted,p=0.6,list = F) # p -> the percentage of data that
goes to training. #list = F -> the result will be in a matrix and not a list
train.diabetes<-diabetesDF[trainIDs,]
test.diabetes<-diabetesDF[-trainIDs,]
```

מריצים פונקציית re-process לניקוי הטקסט

```
diabetesCorpus.train <- preprocess(train.diabetes$diag.text)
for(i in 1:5){
  print(diabetesCorpus.train[[i]][1])
}
```

יצירת DTM ל train.

```
train.dtm <- DocumentTermMatrix(diabetesCorpus.train)
```

```
inspect(train.dtm)
```

```
<<DocumentTermMatrix (documents: 5100, terms: 896)>>
Non-/sparse entries: 63320/4506280
Sparsity : 99%
Maximal term length: 22
Weighting : term frequency (tf)
Sample :
```

```
diabetesCorpus.test <-
```

```
preprocess(test.diabetes$diag.text)
```

Docs	complic	diabet	malign	mellitus	mention	state	type	uncontrol	unspecifi	without
1030	3	1	0	1	2	1	2	1	2	2
1124	3	2	0	2	1	1	2	1	2	1
1334	2	1	0	0	1	1	2	1	2	1
1633	3	2	0	2	0	0	0	0	3	0
2008	2	0	0	0	1	0	0	0	2	1
2786	2	0	0	0	1	0	0	0	2	1
2802	3	2	0	2	1	1	2	1	2	1
3727	3	2	0	2	1	1	2	1	2	1
3868	3	3	0	3	1	1	2	1	2	1
976	3	3	0	3	1	0	2	1	2	1

```
test.dtm <- DocumentTermMatrix(diabetesCorpus.test, control = list(dictionary=Terms(train.dtm)))
```

```
inspect(test.dtm)
```

עושים זאת גם על ה test בשימוש המילים שמופיעות ב train.

כעת נבנה מודל חיזוי. נבנה זאת בפונקציה שנוכל להשתמש בה הרבה פעמים.

מקבלת מטריצה של train ו test ומשתמשת לצורך זה בפונקציה cv.glmnet.

```
lassoPredict <- function(train.mat, test.mat){  
  cv <- cv.glmnet(train.mat,  
    y = as.factor(train.diabetes$readmitted),  
    alpha=1,  
    family='binomial',  
    nfolds=10,  
    intercept=F,  
    type.measure = 'class')
```

אחרי שבנינו את מודל cv נבצע predict- שימוש במודל כדי לחזות האם בן אדם יתאשפז שוב עם נתונים שמודל לא ראה.

```
  preds <- predict(cv, test.mat,  
    type = 'class', s=cv$lambda.1se)  
  # נבנה confusionMatrix להערכת ביצועים. אחד מהמדדים זה accuracy. מדד [1] overall ייתן את זה  
  cm <- confusionMatrix(as.factor(preds),  
    as.factor(test.diabetes$readmitted))  
  return(cm$overall[1])  
}
```

ניצור פונקציה נוספת כדי לבחון כמה מודלים שונים וכמה פונקציות שונות. נשתמש בפונקציה שנקראת svmPredict מטריצה של train ו svmPredict

```
svmPredict <- function(train.mat, test.mat){  
  sv <- svm(train.mat, y= as.factor(train.diabetes$readmitted), gamma = 0.1)  
  # נריץ אותו על נתוני test ונקבל את החיזוי  
  preds <- predict(sv, test.mat) #na.action = na.exclude  
  # נבנה confusionMatrix עם תוצאות החיזוי לעומת התוצאות בפועל ונקבל את ה accuracy.  
  cm <- confusionMatrix(as.factor(preds),  
    as.factor(test.diabetes$readmitted))  
  return(cm$overall[1])}
```

כעת, נריץ את המודלים ונראה מה מקבלים. כדי לקבל את הפונקציות הנ"ל, נרצה לקבל מטריצה. נעשה גם ל test.

```
train.matrix<-Matrix(as.matrix(train.dtm), sparse = T)  
test.matrix<-Matrix(as.matrix(test.dtm), sparse = T)  
set.seed(1)  
lassoPredict(train.matrix, test.matrix)
```

יצא שיש 67% דיוק.

lasso עבור מודל מספרי (מהעמודה הראשונה עד ה 132 מתוך 137 קיימות) פה נקבל דיוק- 0.73%

```
lassoPredict(as.matrix(train.diabetes[,1:132]), as.matrix(test.diabetes[,1:132]))
```

נרצה לבדוק את מודל svm ופה נקבל 0.67%

```
svmPredict(as.matrix(train.diabetes[,1:132]), as.matrix(test.diabetes[,1:132]))
```

סיימנו עם ה bow והמשתנים האחרים. כעת, נרצה לעשות bow בשילוב משתנים האחרים (טקסט + מספרים).

```
cbind(c(1,2),c(3,4))
```

```
set.seed(1)
```

ניתן לראות שהמודל ממש השתפר ורמת הדיוק בו היא 0.74%

```
lassoPredict(cbind(train.matrix, as.matrix(train.diabetes[,1:132])),
```

```
cbind(test.matrix, as.matrix(test.diabetes[,1:132])))
```

נבדוק גם ב svm לראות האם הביצועים טובים יותר ונקבל דיוק של 0.65%, לכן עדיף lassoPredict

```
svmPredict(cbind(train.matrix, as.matrix(train.diabetes[,1:132])),
```

```
cbind(test.matrix, as.matrix(test.diabetes[,1:132])))
```

כעת נבחן את text2vec.

לא נשתמש ב dtm, נלך על data.frame נקי. לכן דבר ראשון צריך ליצור corpus נקי וליצור df.

```
train.diabetes$diag.text <- as.data.frame(diabetesCorpus.train)[,2]
```

```
test.diabetes$diag.text <- as.data.frame(diabetesCorpus.test)[,2]
```

מייצרים איטרטור (עושים את זה טכנית כי זה מה שהאלגוריתם דורש)

```
it <- itoken(train.diabetes$diag.text)
```

```
v <- create_vocabulary(it)
```

```
vectorizer <- vocab_vectorizer(v)
```

נייצר מטריצת co-occurrence matrix:

```
tcm <- create_tcm(it, vectorizer, skip_grams_window = 5L)
```

```
glove <- GlobalVectors$new(rank = 100, x_max = 10)
```

```
word_context <- glove$fit_transform(tcm, n_iter = 100)
```

```
word_vectors <- glove$components + t(word_context)
```

```
word_vectors[1:100,1:10]
```

```
dim(word_vectors)
```

ייצוג וקטורי עבור כל מילה:

acn	-1.601130e-01	-4.090215e-01	1.037993e-01	0.2943963074	0.4942395305	-0.495964460	-0.2456333900
advic	-2.887335e-01	1.140684e-01	-4.467428e-01	0.3727569712	-0.3728738753	0.092723947	0.2662155678
ainhum	3.174099e-01	1.970703e-01	1.458867e-02	0.3421310399	-0.1498069095	-0.423764237	-0.0181039034
alveolar	-3.333859e-01	2.490501e-01	-1.498218e-01	-0.1763381875	-0.3270711843	0.151688303	-0.3853711101
amput	-5.403534e-01	4.546458e-01	1.749440e-01	-0.0510975825	0.4006026517	-0.353852815	0.0491340988
analept	-4.398698e-01	4.380016e-01	-2.578340e-01	-0.1685767385	0.2990989445	0.200925635	-0.2880774669
ani	1.504780e-01	-2.179064e-01	3.541670e-01	-0.5755776043	-0.0656123250	-0.367347017	-0.0775898623
antacid	2.011399e-01	3.857703e-01	-2.781088e-01	-0.5201285169	0.2985208011	-0.210600539	0.4594832253
antiallerg	1.245433e-01	-3.411388e-01	3.632539e-01	-0.4861119462	-0.0506504331	0.351345721	-0.4129614322
antiemet	-3.999080e-01	-8.848861e-02	2.442316e-01	0.5771408406	-0.3058140542	0.332194363	-0.4924777977
	[,22]	[,23]	[,24]	[,25]	[,26]	[,27]	[,28]

נניצר dtm- ב train יש 5100 חולים, נרצה לייצר וקטורים ממוצעים

```
train.dtm <- create_dtm(it, vectorizer)
```

```
dim(train.dtm)
```

נהפוך את זה מ word2vec ל text2vec. ה dim של המטריצה יהיה 5100 על 100- זה אומר שעבור כל מסמך אני מייצגת אותו על ידי וקטור באורך 100.

```
train.dtm <- as.matrix((train.dtm/Matrix::rowSums(train.dtm)) %*% t(word_vectors))
```

```
test.it <- itoken(test.diabetes$diag.text)
```

```
test.dtm <- create_dtm(test.it, vectorizer) # vectorizer contains train vocabulary
```

```
test.dtm <- as.matrix((test.dtm/Matrix::rowSums(test.dtm)) %*% t(word_vectors))
```

נריץ שוב ונראה שעשיו הייצוג הוא 3400 על 100.

```
dim(test.dtm)
```

נריץ אותו הדבר על lassoPredict- המטריצה בנויה מהתוצר של text2vec. קיבלנו 0.66% דיוק.

```
set.seed(1)
```

```
lassoPredict(train.dtm, test.dtm)
```

נעשה את זה גם על svmPredict, הדיוק הינו 0.65%.

```
svmPredict(train.dtm, test.dtm)
```

```
newMatrix <- as.matrix((DTM/rowSums(DTM)) %*% t(WVEC)) # %*% dot product
```

Text Representaion-

כיצד לייצג טקסט כדי שהמחש יבין אותו.

שיטות:

1. **Bag of Words** - בניית dtm שאין חשיבות לסדר בין המילים, מספיק שהיא רק תהיה קיימת. ראינו כמה שיטות לבניית המטריצה.

a. Term frequency - נספור כמה פעמים מילה מופיעה במסמך מסוים.

b. tf-idf - נותנת משקל גבוה לפי מספר הפעמים שהמילה מופיעה וגם למילים שהן נדירות בקורפוס. למשל מילה שמופיעה רק ב 10 מסמכים מתוך 100 מסמכים.

2. **Word Embedding**

a. Word2Vec - מילה מיוצגת על ידי הקונטקסט שלה- ניתן להחליף את word2vec ל text2vec.

Bag of Words - Term Frequency

Document 1: John likes to watch movies. Mary likes movies too.

Document 2: John also likes to watch football games.

בכל תא נכתוב כמה פעמים מופיעה המילה במסמך.

Doc	John	likes	to	watch	movies	Mary	too	also	football	games
1	1	2	1	1	2	1	1	0	0	0
2	1	1	1	1	0	0	0	1	1	1

Bag of N-grams

בגלל ש BOW מאבד את הקונטקסט זה לשמור על המילים צמודות ואז במקום להגדיר במטריצה כל עמודה כמילה בודדת, ניתן להגדיר כל צמד מילים כעמודה.

Unigram- [let] [me] [explain] [with] [an] [example]

Bigram- [let me] [me explain] [explain with] [with an] [an example]

Trigram- [let me explain] [me explain with] [explain with an] [with an example]

זה חשוב עבור הקונטקסט- מילים בעלות משמעות כמו high school שמקבל משמעות אחרת לעומת high. הרבה פעמים בונים מטריצה גם מ unigrams וגם מ bigrams ובוחנים האם האלגוריתם שלנו משתפר או לא בשינויים הללו. צריך לדעת איך לקחת משפט ולפרק אותו לפי האופציות השונות.

TF-IDF- Term Frequency

שמגדירים dtm יש פרמטר של משקל שיכולים להגדיר- איך רוצים שהמטריצה תיווצר (binary- כמה פעמים המילה מופיעה). משקל tf-idf נותן עוצמה למילים יותר נדירות בקורפוס ומאפס מילים שחוזרות על עצמן בכל המסמכים.

כיצד לחשב זאת? $TFIDF = TF * IDF$

נדירות של מילה מסוימת במסמך $TF = (\text{term occurrences in a document}) / (\text{total terms in the document})$

נדירות של מילה ביחס לקורפוס $IDF = \log(\text{total documents in corpus} / \text{number of documents with term } t \text{ in it})$

Number of occurrences of 'kindle' in doc#1 = 5

Number of terms in doc#1 = 200

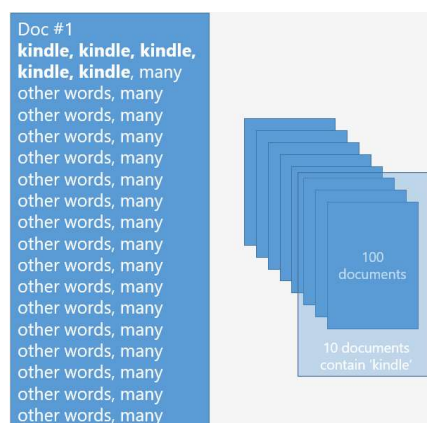
TF = $5/200 = 0.025$

Total documents = 100

Number of documents with the word 'kindle' = 10

IDF = $\log(100/10) = 3.32$

TF*IDF = $0.025 \times 3.32 = 0.083$



לסיכום:

ספריית tm שנתנה לנו יכולת לייצר vcorpus ו dtm על מנת לייצג מילים כ word2vec נתנה לנו אפשרות לייצר ייצוג וקטורי לכל מילה.

Text preprocessing-

הרבה פעמים מקבלים את הטקסטים בצורה שאינה נקיה ולא מוכנה לניתוח.

TM Function	Description	Before	After
<u>tolower</u>	Makes all text lowercase	Starbuck's is from Seattle.	starbuck's is from seattle.
<u>removePunctuation</u>	Removes punctuation like periods and exclamation points	Watch out! That coffee is going to spill!	Watch out That coffee is going to spill
<u>stripWhitespace</u>	Removes tabs, extra spaces	I like coffee.	I like coffee.
<u>removeNumbers</u>	Removes numerals	I drank 4 cups of coffee 2 days ago.	I drank cups of coffee days ago.
<u>removeWords</u>	Removes specific words (e.g. he, and, were & she) defined by the data scientists	The coffee house and barista he visited were nice, she said hello.	The coffee house barista visited nice, said hello.
<u>stemDocument</u>	Reduces prefixes and suffixes on words making term aggregation easier.	Transforming words to do text mining applications is often needed.	Transform word to do text mine applic is often needed.

יש אוסף ניקיונות בסיסיים שחוזרים על עצמם להרבה משימות- נרצה להביא את הטקסט שלנו למצב שכמה שיותר מילים חוזרות על עצמן (עם אותה המשמעות).

סדר שאינו נכון לניקוי:

1. removePunctuation לפני הסרת אימוג'ים (שנותן טקסט לפני אימוג'ים). אם נוריד קודם כל סימני פיסוק זה יפספס את הפואנטה.
2. removeWords לפני tolower.

Stem vs. Root vs. Lemma

Stem	Root	Lemma
The part of the word that is common to all its inflected variants	Primary lexical unit of a word, cannot be reduced into smaller constituents	The canonical form, dictionary form, or citation form of a set of words
stem + affixes = valid word	doesn't have to be a word	valid word
Example	Example	Example
Words: produce, production	Word: interrupt	Words: fly, flies, flew, flying
Stem = produc	Root = rupt	Lemma = fly

מה ההבדל בין stem ו lemma?

-stem מבצע חיתוך של המילה, אלו חוקים שהם מאוד טכניים של איך לחתוך מילה. אם יש לי למשל es או s בסוף יש הגדרות לאיך חותכים את המילה. הקיצוץ הזה מביא למילה שהיא לא בהכרח מילה חוקית בשפה, כמו בדוגמא. אבל היא מספקת מהבחינה שהיא מספיק ברורה שנבין מה כוונתה.

-Lemma תהליך שמביא את המילה לצורת הבסיס שלה, הוא מביא אותה תמיד לצורה שקיימת במילון.

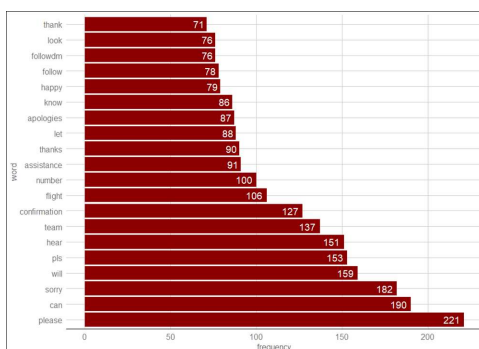
אחד החסרונות של Lemma- הוא זמן ההרצה. שלוקח יותר מה stem. הוא הולך ומחפש במילון הדיפולטי שמוגדר לו.

לסיכום:

ספריות שנשתמש בהן לניקיונות:

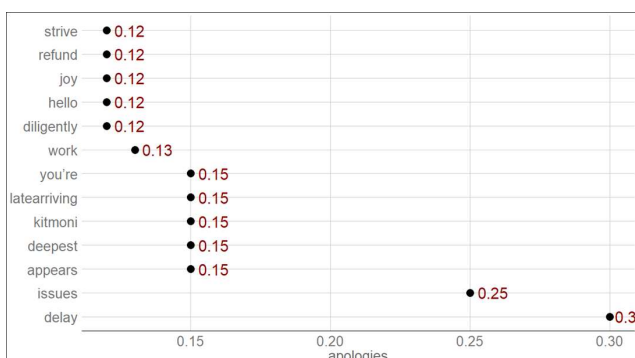
- tm- מספק את כלל הפונקציות לניקוי
- gsub- עבור find and replace, לדוגמא החלפת http
- textstem- לביצוע stemming

Visualization-

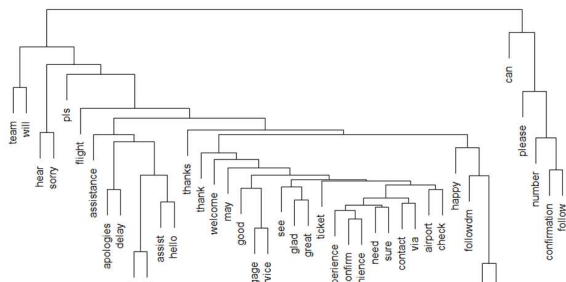


אחרי ייצוג וניקוי טקסט למדנו כיצד לבצע ויזואליזציות להבין מה עומד בפנינו ואילו מילים נפוצות ורעשים בניתוח.

Bar Plot of word frequencies - ויזואליזציה בסיסית להבין בקורפוס איזה אוסף מילים הכי נפוץ.



Word Associations – cont מוצא אסוציאציה בין מילים מסוימות, לדוגמא איזה מילים מופיעות הרבה. ככל שהציון גדול יותר ככה המילה מופיעה יותר פעמים.



Word Clusters: Hierarchical Dendrograms

ייצוג שמאפשר לראות קרבה בין מילים, לדוגמא מילה מסויימת שמופיעה הרבה פעמים עם מילה אחרת.

כך ניתן לקבץ ביחד מילים שמופיעות הרבה ביחד.

דבר נוסף שראינו, פונקצית sparse. מטריצה שיש הרבה אפסים בתוכה.

Word Cloud / Tag Cloud

ויזואליזציה נפוצה שמדגישה את המילים לפי החשיבות שלהן.

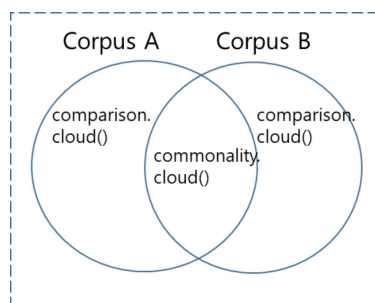
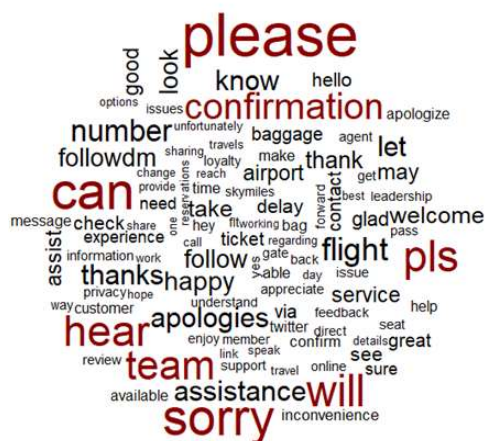
לוקחת את כל המילים בקורפוס סופרת כמה פעמים מילה מופיעה בסה"כ

ולפי זה נקבע הגודל שלה.

ניתן להרחיב את הרעיון וליצור

commonality/comparison שנרצה

להשוות בין שני קורפוסים.



החסרונות של commonality cloud - לא נותן לנו את ההפרש בין כמות הפעמים שמילה מסוימת מופיעה במקום 1 לעומת מקום אחר. למשל אם יש מילה please כ- 50 פעמים במקום 1 וכ- 200 פעמים במקום אחר - זה יופיע כגודל אחיד (לפי החיתוך) ולא נדע את ההפרשים בין 2 המסמכים.

החסרונות של comparison cloud - אם מילה מופיעה ב- 2 הקורפוסים בטקסט, ויזואלית היא תופיע רק בקורפוס 1.

Sentiment scoring based on lexicons

למדנו איך אנחנו מנתחים רגשות חיוביים או שליליים באמצעות אלגוריתם שמבוסס על חוקים לא באמצעות קלסיפיקציה.

Polarity: Simple Sentiment Scoring

Example: "Sentiment analysis in R is good yet challenging"

Word	Score
Good	+1
Challenging	-1

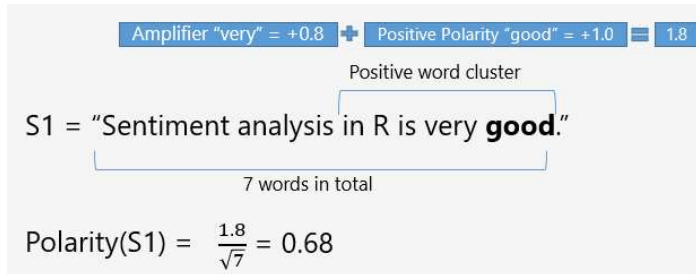
מקבלים משפט ונרצה לדעת מה הסנטימנט, האם חיובי או שלילי. לכל מילה נעבור ונראה האם היא קיימת בלקסיקון ועם איזה ציון. ובסה"כ הציון הוא 0.

Valence shifters

- Negator - flips the sign of a polarized word (e.g. "I do not like it")
- Amplifier (intensifier) - increases the impact of a polarized word (e.g. "I really like it")
- De-amplifier (downtoner) - reduces the impact of a polarized word (e.g. "I hardly like it")

יש מילים שמשנות את משמעות המשפט (חיובי או שלילי) ולכן חשוב לא לנקות stopwords שמנתחים טקסט על ידי sentiment scoring.

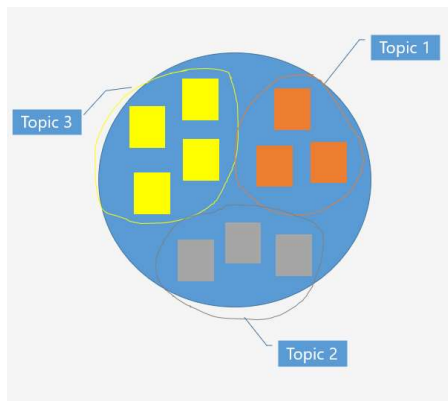
Qdap's polarity function



למשל אם יש משפט- יש לחשב את הציון (polarity), נעבור מילה מילה ונחפש מילה שנמצאת בלקסיקון שלנו. מצאנו את המילה good. ניקח את הסביבה הקרובה ובדוק האם יש משהו שיכול לשנות את המשמעות של המילה או לשנות את הניקוד שלה. very יעלה את הציון שלה.

הדבר החשוב זה לדעת שעושים ניקיונות שונים בחישובים מסוג זה.

Unsupervised learning – Topic Modeling and Cluster Analysis



2 אלגוריתמים מרכזיים שנלמדו.

supervised- יש סט נתונים שידוע לנו בהם שדה המטרה.

unsupervised- אין תיוג לנתונים.

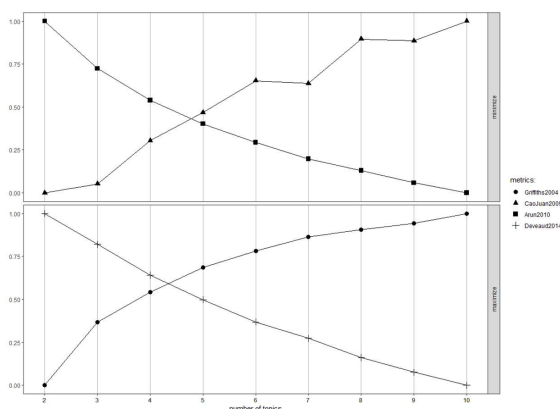
Topic Modeling Goal- אוסף מסמכים שמחולקים לנושאים.

האלגוריתם הזה מתבסס על שיטות הסתברותיות. האלגוריתם נותן לכל מסך חישוב כמה ממנו הוא כל אחד מה topics. מכיוון שמסמך יכול לדבר על הרבה נושאים.

האלגוריתם מחשב עבור כל נושא מה המילים שמאפיינות אותו ולכל מסמך את ההסתברות שמסמך שייך לנושא.

Topic 1	Topic 2	Topic 3
library	number	HR
skmeans	cluster	work
dtm	ensemble	recruiting
corpus	K-Means	team
weight	Spherical	experience
cluster	weight	educate

Document	Topic 1	Topic 2	Topic 3
1	0.31	0.29	0.40
2	0.34	0.23	0.43
3	0.31	0.25	0.44
4	0.35	0.36	0.29
5	0.28	0.46	0.26
6	0.33	0.30	0.37



איך נדע לכמה נושאים לחלק? זה לא מדע מדויק, מחלקים ובודקים.

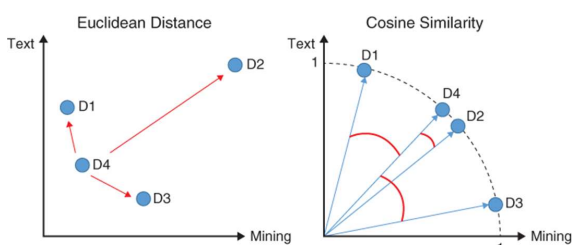
וגם לפי מדדים מקובלים שעוזרים לקבל החלטה לכמה נושאים לחלק (max | min).

	Term 1	Term 2	Term 3	Term 4	...	Term p
Document 1						
Document 2						
Document 3						
Document 4						
Document 5						
...						
Document n						

Topic Modeling - כמה פעמים מילה מופיעה במסמך מסוים. ושיוך לנושא.

	Term 1	Term 2	Term 3	Term 4	...	Term p
Document 1						
Document 2						
Document 3						
Document 4						
Document 5						
...						
Document n						

Cluster Analysis - מודד מרחקים בין הוקטורים של כל מסמך. אם מילה מסוימת מופיעה בשני מקומות.



A simple bag-of-words approach

יש 2 שיטות לחישוב המרחקים.

שיטה אוקלידית ושיטה למרחק שבין זוויות.

אם הזווית היא 0 אין דמיון ואם יוצא 1 אז יש דמיון.

האלגוריתם שמשמשים בו זה spherical k means

Supervised learning – Document Classification

Supervised

- ✓ Outcome (y) is **known**
- ✓ Goal: **prediction**
- ✓ Examples: linear regression, logistic regression, classification trees, neural networks

Unsupervised

- ✓ Outcome (y) is **unknown**
- ✓ Goal: find **patterns** or **groups**
- ✓ Examples: cluster analysis, factor analysis, topic modeling

Supervised/ Unsupervised Learning

ההבדלים בין שתי השיטות.

Text Classification

ישנם מסמכים, עבור כל מסמך יש את התיוג שלו (class זה הסיווג), מייצגים אותו כ BOW ובונים dtm עם עמודת class.

נפעיל על זה אלגוריתם מסוים שבונה מודל.

אחרי שיש מודל נחשב משקלים מסוימים ואלו

שהשפיעו על התוצאה. נבחנו כמה שיטות לצורך

זה.lasso/svm

